



UNIVERSITÀ DEGLI STUDI DI GENOVA
Scuola Politecnica — Ingegneria Elettronica Magistrale

**Studio comparativo tra il modello reale
e il gemello digitale di un sistema
di condizionamento termico
basato su cella di Peltier**

CORSO: CYBER PHYSICAL SYSTEMS — 114757

Lorenzo Raspaolo
Matricola: S5016651

Anno Accademico 2025/2026
20 giugno 2026

Sommario

I sistemi cyber-fisici (CPS) integrano dinamica fisica continua e logica di controllo discreta in un'unica architettura che opera sul mondo reale. Questo lavoro presenta il progetto e la validazione di un *gemello digitale* per un sistema di condizionamento termico a effetto Peltier: l'obiettivo è riprodurre fedelmente nel dominio virtuale il comportamento termico e di controllo dell'impianto fisico, consentendo analisi predittive e verifica di proprietà di sicurezza senza mettere a rischio l'hardware.

Il sistema fisico è composto da una cella termoelettrica al tellururo di bismuto (Bi_2Te_3 , lato 16 mm, identificata sperimentalmente con $R = 2,04\,\Omega$ e $\alpha = 0,0152\,\text{V K}^{-1}$), da sensori PT100 interfacciati tramite convertitore MAX31865, e da un firmware PID eseguito su Arduino UNO Q (core STM32U585). Il gemello digitale è realizzato in MATLAB come simulazione a parametri concentrati con integrazione di Eulero a passo 1 s (stesso intervallo del controllore Arduino, in modo da riprodurre esattamente la discretizzazione del PID). Validato su un profilo multi-setpoint di 6840 s (tre gradini $\text{SP} = 22\,^\circ\text{C}$, $20\,^\circ\text{C}$, $18\,^\circ\text{C}$ alternati a fasi di riposo), il gemello raggiunge un errore MAPE (*Mean Absolute Percentage Error*, scarto percentuale medio assoluto) di 3,2% sulla temperatura del lato freddo, corrispondente a un indice di fedeltà del 96,8%.

La macchina a stati del firmware è verificata formalmente con UPPAAL: quattro proprietà di safety e liveness sono dimostrate sul modello a cinque stati, inclusa la garanzia che un'eccedenza di temperatura sul lato caldo provochi immediatamente la transizione in `THERMAL_FAULT`.

Indice

1	Introduzione	6
1.1	Contesto e motivazione	6
1.2	Obiettivi	6
1.3	Deviazioni rispetto alla proposta iniziale	7
2	Architettura del sistema	8
2.1	Schema a blocchi	8
2.2	Distinta dei componenti	9
2.3	Mappa fisico-cyber	10
3	Modellazione fisico-matematica	12
3.1	Equazioni della cella termoelettrica	13
3.2	Dinamica termica delle piastre e dei dissipatori	13
3.3	Scambio termico con l'ambiente	14
3.4	Ipotesi semplificative	14
4	Identificazione sperimentale della cella	16
4.1	Il problema dell'identificazione	16
4.2	Procedure di misura non distruttive	16
4.3	Risultati e identificazione del modello	18
4.4	Ricalibrazione del gemello digitale	18
4.5	Inviluppo operativo di sicurezza	18
5	Il gemello digitale in Simulink	20
5.1	Architettura del modello	21
5.2	Nucleo termoelettrico	22
5.3	Generazione del setpoint: profilo a fasi	22
5.4	Architettura di controllo	23
5.5	Stadio di potenza	23
5.6	Non-idealità di misura e attuazione	24
5.7	Risultati di simulazione	24

6	Realizzazione hardware	26
6.1	La piattaforma di calcolo: Arduino UNO Q	26
6.2	Catena di acquisizione delle temperature	26
6.3	Stadio di potenza	27
6.4	Alimentazione e protezioni	27
6.5	Assemblaggio termo-meccanico	28
7	Firmware embedded	29
7.1	Architettura software	29
7.2	Generazione PWM con timer Zephyr	29
7.3	Macchina a stati di supervisione	30
7.4	Controllore PID	31
7.5	Gestione dei guasti e safety	32
7.6	Telemetria	33
8	Verifica formale in UPPAAL	34
8.1	Automati temporizzati: richiami	34
8.2	Modello del sistema	35
8.3	Proprietà verificate e risultati	37
8.4	Iterazioni di debug del modello	38
8.5	Impatto sul progetto del firmware	40
9	Validazione sperimentale e confronto modello–reale	41
9.1	Protocollo sperimentale	41
9.2	Risultati	42
9.3	Metriche di confronto	43
9.4	Analisi delle discrepanze	44
9.5	Raffinamento del modello	44
10	Conclusioni e sviluppi futuri	46
10.1	Sintesi dei risultati	46
10.2	Limiti attuali	46
10.3	Sviluppi futuri	47
A	Listato del firmware	48
B	Modello UPPAAL	54

Elenco delle figure

2.1	Schema a blocchi del sistema di condizionamento termico. . .	8
3.1	Circuito termico equivalente della cella Peltier con le due masse termiche, le conduttanze verso l'ambiente e le equazioni delle ODEs implementate nel gemello digitale.	12
5.1	Vista completa del modello Simulink <code>Modello_Cella_Peltier.slx</code> . I principali sottosistemi sono visibili: condizioni ambientali (alto sinistra), dinamica termica e masse (centro sinistra), fisici Peltier + driver (centro), setpoint + controllore PID (centro destra), logger (destra). Le figure successive mostrano i sottosistemi in dettaglio.	20
5.2	Sottosistema Condizioni Ambientali e Scambi Convettivi : modella l'irraggiamento termico verso la testa a vuoto, l'isolamento, e gli scambi convettivi tra le piastre e l'aria. Il blocco Chip da Raffreddare (sinistra) introduce un carico termico esterno aggiuntivo con feed-forward.	21
5.3	Nucleo del modello Simulink: dinamica termica e masse (lato sinistro), generazione del setpoint multi-fase con rampe (centro) e controllore PID con saturazione e conversione DAC/PWM (lato destro).	22
5.4	Sottosistemi Motore Termoelettrico (fisica della cella Peltier, a sinistra) e Stadio di Potenza e Driver con il blocco hardware TPS54618 (a destra). Il parametro $V_{in,max} = 6$ corrisponde alla tensione di alimentazione del banco.	24
5.5	Confronto tra simulazione (gemello digitale, integrazione Eulero 1s) e dati sperimentali reali (CSV, test 6840 s). In alto: temperature lato freddo e lato caldo con il profilo di setpoint. In basso: output PID simulato vs misurato. La MAPE sulle fasi attive è $\approx 3,2\%$ (match 96,8%).	25
6.1	Banco di prova assemblato. Le PT100 sono fissate con nastro adesivo termico sulle superfici dei dissipatori.	28

7.1	Schema del controllore PID in modalità REVERSE: errore positivo quando $T_{\text{fredda}} > SP$ (cella troppo calda).	32
8.1	Template Cella_Peltier nel GUI di UPPAAL: FSM a cinque stati con guardie, sincronizzazioni e il clock x per la DEAD_BAND.	36
8.2	Template Pulsantiera (supervisore): unica location IDLE con tutte le transizioni verso l'ambiente non deterministico. Le guardie anti-loop ($T_{\text{calda}} \neq 65$, $pt100_ok = \text{true}$) evitano i livelock da τ -transizioni idempotenti.	37
8.3	Pannello <i>Verifier</i> di UPPAAL con le quattro proprietà verificate (pallino verde = soddisfatta). Il tempo di verifica è inferiore a 1 s per ciascuna.	38
9.1	Setup sperimentale completo: laptop con terminale seriale e log CSV (sinistra), oscilloscopio RIGOL per il monitoraggio del segnale PWM (centro) e alimentatore da banco regolato a 6 V (destra). La scheda Arduino UNO Q e i moduli MAX31865 sono visibili al centro del banco, collegati alla cella Peltier.	42
9.2	Profilo multi-setpoint del test sperimentale: sei fasi con tre gradini attivi e due periodi di riposo.	42
9.3	Confronto gemello digitale vs impianto reale: temperature lato freddo e lato caldo (pannello superiore) e errore di simulazione ΔT (pannello inferiore). MAPE globale 3,2 %, match 96,8 %.	43
9.4	Segnale di controllo PID simulato vs reale (pannello superiore) e tensione/corrente della cella simulate (pannello inferiore).	43
B.1	Template del modello UPPAAL (si veda il Capitolo 8 per la descrizione dettagliata).	55

Elenco delle tabelle

2.1	Distinta dei componenti (BOM) del banco di prova.	10
2.2	Mappa tra i sottosistemi fisici e i loro corrispondenti nel do- minio cyber.	11
4.1	Confronto tra parametri identificati e valori di datasheet. . . .	18
4.2	Inviluppo operativo di sicurezza della cella.	19
5.1	Metriche di confronto simulazione vs misura reale (test 6840 s). .	25
7.1	Allineamento firmware–UPPAAL: stati, transizioni e soglie. . .	31
8.1	Proprietà UPPAAL verificate sul modello finale.	37
9.1	Metriche di confronto gemello digitale vs impianto reale (test 6840 s, fasi attive SP1+SP2+SP3).	44

Capitolo 1

Introduzione

1.1 Contesto e motivazione

Un *Cyber-Physical System* (CPS) è un'architettura in cui componenti computazionali e di rete interagiscono in modo stretto con processi fisici. Sensori raccolgono grandezze reali, algoritmi elaborano le misure e attuatori retroagiscono sull'impianto: il ciclo continuo tra dominio fisico e dominio digitale distingue i CPS dai sistemi embedded tradizionali e li rende particolarmente adatti ad applicazioni di monitoraggio, controllo e manutenzione predittiva.

Il controllo di temperatura è uno dei campi applicativi più rappresentativi di questa classe di sistemi. Le celle di Peltier, dispositivi a stato solido basati sull'effetto termoelettrico di Peltier-Seebeck, vengono impiegate quando si richiedono dimensioni compatte, silenziosità operativa e reversibilità della direzione del flusso termico. Trovano applicazione in refrigerazione di precisione, stabilizzazione termica di laser e sensori, e sistemi di test ambientale su piccola scala.

La costruzione di un gemello digitale (*digital twin*) per tale impianto risponde a due esigenze: da un lato permette di esplorare scenari operativi estremi (sovratemperatura, guasto sensore) senza rischiare danni all'hardware; dall'altro consente la verifica formale delle proprietà di sicurezza della logica di controllo prima del dispiegamento sul target.

1.2 Obiettivi

Il progetto si articola in tre obiettivi principali:

1. **Identificazione sperimentale della cella.** I parametri elettrici e termici della cella termoelettrica montata (resistenza interna R , coefficiente di Seebeck α , conduttanza termica K) vengono misurati in situ con procedure non distruttive e confrontati con i valori di datasheet, poiché il modello fisico esatto non è verificabile a vista.

2. **Gemello digitale in Simulink.** Un modello continuo in tempo reale replica la fisica della cella (polinomi $\alpha(T)$, $R(T)$, $K(T)$ del Bi_2Te_3), le masse termiche dei dissipatori, le non-idealità di misura (rumore, quantizzazione ADC, ritardo PT100) e il controllore PID con gain scheduling. Il modello viene validato confrontando il transitorio simulato con quello dell'impianto reale.
3. **Verifica formale in UPPAAL.** La macchina a stati del firmware (INIT, STANDBY, ACTIVE_CONTROL, DEAD_BAND, THERMAL_FAULT) è modellata come automa temporizzato e verificata contro proprietà di assenza di deadlock, raggiungibilità degli stati critici e garanzie di safety sulla soglia termica.

Il criterio di successo quantitativo per il gemello digitale è un errore RMS tra temperatura simulata e misurata inferiore a 3°C nel transitorio nominale. Per la verifica formale il criterio è il superamento di tutte le proprietà definite senza controesempi.

1.3 Deviazioni rispetto alla proposta iniziale

La proposta presentata al docente prevedeva alcune scelte hardware che, in fase di realizzazione, hanno richiesto adattamenti.

- **PT100 al posto di PT1000.** I sensori di temperatura impiegati sono PT100 ($100\ \Omega$ a 0°C) e non PT1000 come indicato nella proposta. I moduli MAX31865 sono stati configurati in configurazione a **due fili** con $R_{\text{ref}} = 430\ \Omega$. Le specifiche di risoluzione ($0,03125^\circ\text{C}$ per passo ADC a 15 bit) e di accuratezza rimangono adeguate agli scopi del progetto.
- **Arduino UNO Q come piattaforma STM32U585.** La scheda Arduino UNO Q (codice ABX00162) integra un'architettura *dual-brain*: un microprocessore Qualcomm gestisce la parte Linux, mentre il core STM32U585 esegue il firmware embedded in tempo reale. Questa soluzione soddisfa il requisito concordato di utilizzare il microcontrollore STM32U585 con sistema operativo Zephyr RTOS.
- **Stadio di potenza a MOSFET singolo.** La proposta menzionava il convertitore buck TPS54618 come opzione “da verificarne la fattibilità realizzativa”. A seguito dell'analisi, si è scelto di pilotare direttamente la cella con un MOSFET N-channel AOD4184A in switching a $100\ \text{Hz}$, accettando la natura unidirezionale del controllo (solo raffreddamento attivo). Questa scelta semplifica lo stadio di potenza riducendo il rischio di danneggiamento della cella e rimane nei limiti di sicurezza ($V \leq 6\ \text{V}$, $I \leq 3\ \text{A}$).

Capitolo 2

Architettura del sistema

Il sistema si articola in tre sottosistemi interconnessi: il banco fisico (cella Peltier, stadio di potenza, sensori), il firmware embedded (microcontrollore Arduino UNO Q), e lo strato di telemetria (collegamento seriale verso il PC ospite). La Figura 2.1 ne mostra lo schema a blocchi.

2.1 Schema a blocchi

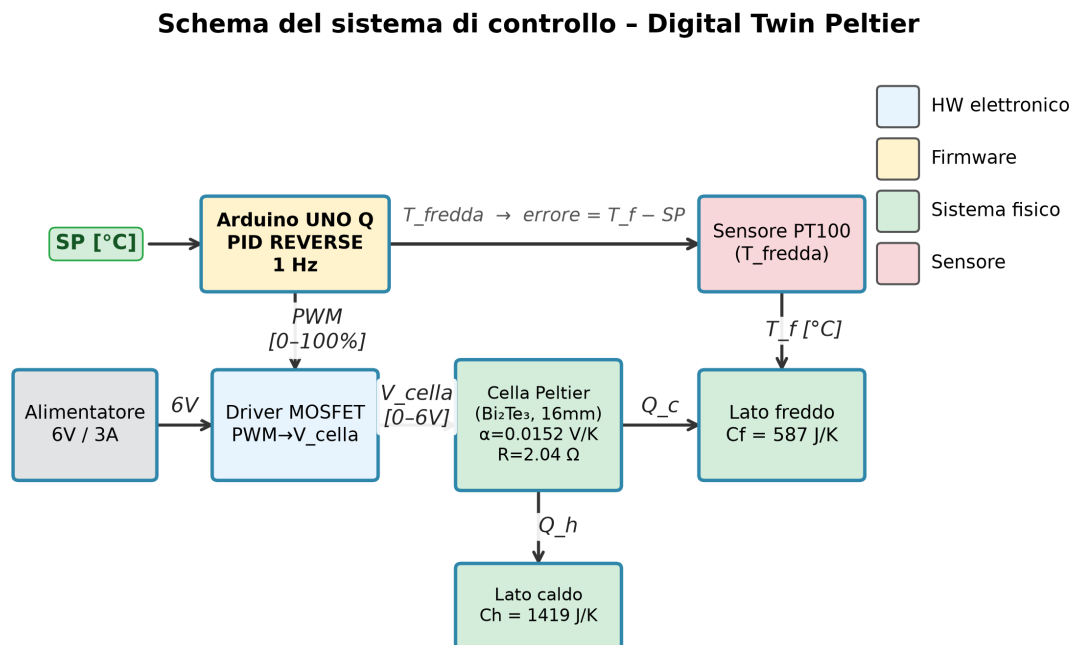


Figura 2.1: Schema a blocchi del sistema di condizionamento termico.

Il **percorso di potenza** parte dall'alimentatore da banco (regolato a $V_{cc} = 6\text{ V}$, $I_{lim} = 3\text{ A}$), attraversa il MOSFET N-channel AOD4184A pilotato in switching a 100 Hz con segnale PWM generato dal microcontrollore, e raggiunge la cella termoelettrica. Il duty cycle controlla la corrente media erogata alla cella, modulando la potenza di pompaggio termico. Sul lato caldo e sul lato freddo sono incollati rispettivamente un dissipatore in alluminio grande ($120 \times 80 \times 10\text{ mm}$) e uno piccolo ($130 \times 30 \times 5\text{ mm}$).

Il **percorso di misura** origina da due sensori PT100 a due fili posizionati sul lato caldo e sul lato freddo della cella. Ogni PT100 è interfacciata tramite un modulo MAX31865, che esegue la conversione RTD→digitale (15 bit, risoluzione $0,03125^\circ\text{C}$) e restituisce la temperatura al microcontrollore tramite bus SPI. La resistenza di riferimento del MAX31865 è stata impostata a $R_{ref} = 430\ \Omega$.

Il **percorso di controllo e telemetria** risiede sul core STM32U585 dell'Arduino UNO Q, che esegue il firmware Zephyr RTOS. Il firmware implementa il controllore PID, la macchina a stati di supervisione e l'invio periodico dei dati di telemetria (setpoint, temperatura misurata, duty cycle, stato FSM) via UART/USB al PC ospite.

2.2 Distinta dei componenti

La Tabella 2.1 elenca i componenti principali del banco di prova con i rispettivi riferimenti.

Tabella 2.1: Distinta dei componenti (BOM) del banco di prova.

Riferimento	Componente	Note
U1	Cella termoelettrica Bi_2Te_3 , lato 16 mm	$R = 2,04 \Omega$, $\alpha = 0,0152 \text{ V K}^{-1}$ (identificati sperimentalmente)
MCU	Arduino UNO Q (ABX00162)	Core STM32U585 (Zephyr RTOS); il coprocessore Qualcomm QCC5181 (Linux) è presente sulla scheda ma non utilizzato in questo progetto
U2, U3	Modulo MAX31865 ($\times 2$)	RTD-to-digital, SPI, $R_{\text{ref}} = 430 \Omega$
RT1, RT2	PT100 a due fili ($\times 2$)	100Ω a 0°C , posizionati lato caldo e lato freddo
Q1	MOSFET AOD4184A (N-ch, TO-252)	$V_{DS,\text{max}} = 40 \text{ V}$, $I_{D,\text{max}} = 50 \text{ A}$, $R_{DS,\text{on}} = 6,6 \text{ m}\Omega$; pilotato a 100 Hz
HS1	Dissipatore lato caldo, Al	$120 \times 80 \times 10 \text{ mm}$; massa termica calcolata $231,1 \text{ J K}^{-1}$
HS2	Dissipatore lato freddo, Al	$130 \times 30 \times 5 \text{ mm}$; massa termica calcolata $52,7 \text{ J K}^{-1}$
PS1	Alimentatore da banco	Impostato a 6 V , $I_{\text{lim}} = 3 \text{ A}$

I valori di massa termica dei dissipatori sono stati calcolati come $C_{\text{th}} = \rho \cdot V \cdot c_p$ con alluminio $\rho = 2700 \text{ kg m}^{-3}$, $c_p = 897 \text{ J kg}^{-1} \text{ K}^{-1}$.

2.3 Mappa fisico-cyber

Il sistema è stato suddiviso in tre livelli di astrazione, ognuno dei quali è affrontato con uno strumento diverso (Tabella 2.2).

Tabella 2.2: Mappa tra i sottosistemi fisici e i loro corrispondenti nel dominio cyber.

Componente fisico	Corrispondente cyber	Strumento
Cella Peltier	Blocco fisico con polinomi $\alpha(T)$, $R(T)$, $K(T)$	Simulink (cap. 5)
Dissipatori (masse termiche)	Integratori termici con convezione e irraggiamento	Simulink (cap. 5)
Sensori PT100 + MAX31865	Rumore additivo, quantizzazione 15 bit, ritardo di campionamento	Simulink (cap. 5)
Controllore PID (firmware)	Blocco PID con gain scheduling	Simulink + UPPAAL (cap. 5, 8)
FSM di supervisione (firmware)	Template Cella_Peltier a 5 stati	UPPAAL (cap. 8)
Arduino UNO Q (loop)	— (gira direttamente sul target)	Firmware (cap. 7)
Ambiente (guasti termici, fault sensore)	Template Pulsantiera non deterministico	UPPAAL (cap. 8)

Il modello del gemello digitale riproduce la fisica del sistema su un orizzonte temporale di 6840s (circa 114 minuti) con un profilo multi-setpoint ($SP1 = 22^\circ\text{C}$, $SP2 = 20^\circ\text{C}$, $SP3 = 18^\circ\text{C}$); la sua fedeltà è valutata nel Capitolo 9 confrontando il transitorio simulato con quello misurato sull'impianto. Il modello UPPAAL astrae la temperatura come variabile intera e si concentra sulle proprietà qualitative della logica di supervisione, indipendentemente dalla dinamica continua. Il firmware è il punto di incontro: la sua macchina a stati deve rispettare sia le proprietà formali dimostrate in UPPAAL sia i vincoli di prestazione osservabili nella simulazione Simulink.

Capitolo 3

Modellazione fisico-matematica

Il comportamento della cella termoelettrica è governato dall'interazione tra tre effetti fisici: l'effetto Peltier (flusso di calore proporzionale alla corrente), l'effetto Joule (dissipazione resistiva) e la conduzione termica (flusso di calore proporzionale a ΔT). Il modello qui presentato segue la formulazione classica di Goldsmid e Rowe [2].

Modello matematico della cella Peltier - Circuito termico equivalente

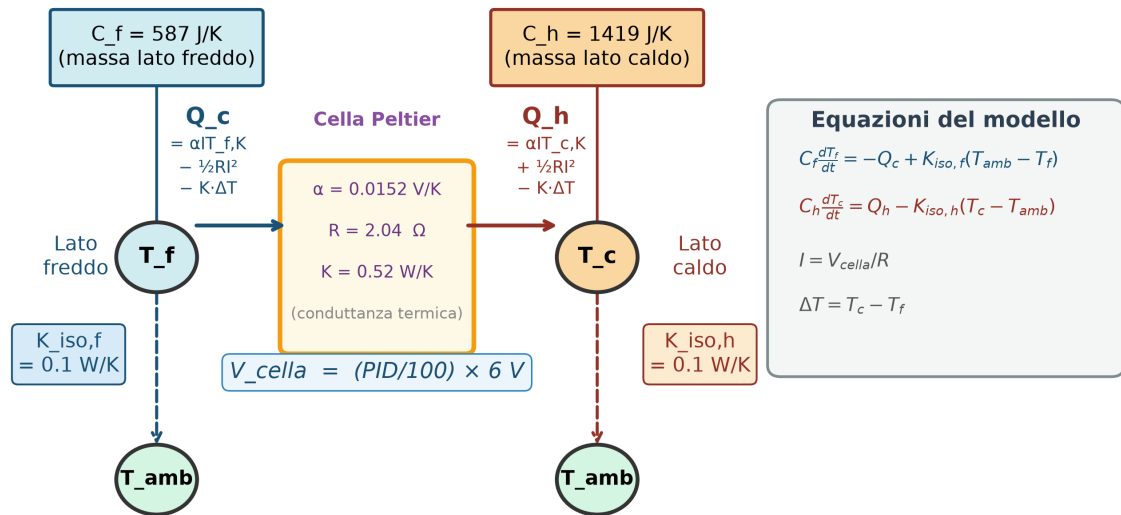


Figura 3.1: Circuito termico equivalente della cella Peltier con le due masse termiche, le conduttanze verso l'ambiente e le equazioni delle ODEs implementate nel gemello digitale.

3.1 Equazioni della cella termoelettrica

Si indicano con T_h e T_c le temperature assolute [K] del lato caldo e del lato freddo, con $\Delta T = T_h - T_c$, con V la tensione applicata e con I la corrente che scorre nella cella.

Corrente di lavoro. L'effetto Seebeck genera una forza contro elettromotrice (f.c.e.m.) pari a $\alpha \Delta T$; la corrente è quindi:

$$I = \frac{V - \alpha \Delta T}{R} \quad (3.1)$$

Potenza termica lato freddo (pompaggio). La potenza estratta dalla piastra fredda è:

$$Q_c = \alpha I T_c - \frac{1}{2} R I^2 - K \Delta T \quad (3.2)$$

Il termine $\alpha I T_c$ è il pompaggio Peltier; $\frac{1}{2} R I^2$ è la metà della dissipazione Joule che refluisce verso il lato freddo; $K \Delta T$ è la conduzione termica dal caldo al freddo attraverso il materiale.

Potenza termica lato caldo (espulsione). La potenza espulsa dalla piastra calda è:

$$Q_h = \alpha I T_h + \frac{1}{2} R I^2 - K \Delta T \quad (3.3)$$

Il bilancio energetico è rispettato: $Q_h = Q_c + V I$.

Parametri dipendenti dalla temperatura. I materiali termoelettrici reali mostrano una dipendenza dei parametri dalla temperatura. Per il Bi_2Te_3 si usano polinomi normalizzati al valore di riferimento a $T_{\text{ref}} = 300 \text{ K}$ (27°C):

$$\alpha(T) = \alpha_0 \cdot f_\alpha(T) \quad (3.4)$$

$$R(T) = R_0 \cdot f_R(T) \quad (3.5)$$

$$K(T) = K_0 \cdot f_K(T) \quad (3.6)$$

dove $\alpha_0 = 0,0152 \text{ V K}^{-1}$ e $R_0 = 2,04 \Omega$ sono i valori identificati sperimentalmente (Capitolo 4); $K_0 \approx 0,52 \text{ W K}^{-1}$ è la conduttanza termica stimata dal transitorio di rilassamento (Sezione 4.2). I fattori di forma f_α , f_R , f_K sono tratti dalle curve standard del Bi_2Te_3 [2] e scalati in modo che $f(T_{\text{ref}}) = 1$.

3.2 Dinamica termica delle piastre e dei dissipatori

I dissipatori costituiscono le masse termiche dominanti del sistema. Il bilancio energetico di ciascuna piastra è:

$$C_{th,h} \frac{dT_h}{dt} = Q_h - Q_{conv,h} - Q_{rad,h} \quad (3.7)$$

$$C_{th,c} \frac{dT_c}{dt} = -Q_c - Q_{conv,c} - Q_{rad,c} \quad (3.8)$$

dove i flussi verso l'ambiente (Q_{conv} , Q_{rad}) sono descritti nella Sezione 3.3.

Capacità termiche. Per entrambi i dissipatori in alluminio ($\rho = 2700 \text{ kg m}^{-3}$, $c_p = 897 \text{ J kg}^{-1} \text{ K}^{-1}$):

$$C_{th,h} = \rho V_h c_p = 2700 \times (0,12 \times 0,08 \times 0,01) \times 897 \approx 232,5 \text{ J K}^{-1} \quad (3.9)$$

$$C_{th,c} = \rho V_c c_p = 2700 \times (0,13 \times 0,03 \times 0,005) \times 897 \approx 47,2 \text{ J K}^{-1} \quad (3.10)$$

Il dissipatore lato caldo, più massiccio, smorzerà le variazioni di T_h su una scala temporale più lunga rispetto al lato freddo, determinando le costanti di tempo dell'impianto.

3.3 Scambio termico con l'ambiente

Convezione naturale. Il flusso convettivo verso l'aria ambientale a temperatura T_{amb} segue la legge di Newton del raffreddamento:

$$Q_{conv} = h A (T - T_{amb}) \quad (3.11)$$

Il coefficiente di convezione h in aria ferma è dell'ordine di da $5 \text{ W m}^{-2} \text{ K}^{-1}$ a $15 \text{ W m}^{-2} \text{ K}^{-1}$; nel modello Simulink si utilizza $h = 10 \text{ W m}^{-2} \text{ K}^{-1}$ come valore nominale di convezione naturale. Le aree di scambio sono quelle delle superfici dei dissipatori ($A_h \approx 0,10 \text{ m}^2$, $A_c \approx 0,04 \text{ m}^2$).

Irraggiamento. Il flusso radiativo è descritto dalla legge di Stefan-Boltzmann:

$$Q_{rad} = \varepsilon \sigma A (T^4 - T_{amb}^4) \quad (3.12)$$

con emissività $\varepsilon = 0,9$ (alluminio anodizzato) e costante di Stefan-Boltzmann $\sigma = 5,67 \times 10^{-8} \text{ W m}^{-2} \text{ K}^{-4}$. Il contributo radiativo risulta trascurabile rispetto alla convezione nelle condizioni operative del banco ($\Delta T < 40 \text{ K}$ rispetto all'ambiente), e viene pertanto incluso nel modello per completezza ma non ottimizzato.

3.4 Ipotesi semplificative

Il modello adotta le seguenti semplificazioni rispetto alla fisica completa dell'impianto:

1. **Temperature omogenee.** Ogni piastra è rappresentata da un'unica temperatura media; la distribuzione spaziale del calore nei dissipatori è trascurata.
2. **Contatto termico ideale.** La resistenza di contatto tra la cella e i dissipatori è assorbita nella calibrazione dei polinomi (Sezione 4.4).
3. **Corrente istantanea.** La dinamica elettrica della cella e del circuito di potenza (costante di tempo $\tau = L/R \sim 10 \mu\text{s}$ per la cella resistiva) è molto più rapida della dinamica termica (costanti di tempo di decine di secondi) e viene trascurata. Nel modello Simulink si inserisce un'inerzia di corrente con $\tau = 1 \text{ ms}$ per riprodurre il ritardo del circuito driver.
4. **PWM mediata.** Il segnale PWM a 100 Hz è rappresentato dal suo valore medio: $V_{\text{eff}} = D \cdot V_{cc}$, dove $D \in [0, 1]$ è il duty cycle imposto dal PID. Questa approssimazione introduce un errore trascurabile sulla dinamica termica ($\tau_{\text{termico}} \gg T_{\text{PWM}}$).
5. **Temperatura ambiente costante.** T_{amb} è fissata al valore di misura iniziale per l'intera simulazione. La dipendenza della convezione dalla posizione e dall'orientamento dei dissipatori non è modellata.

Capitolo 4

Identificazione sperimentale della cella

La costruzione del gemello digitale richiede valori precisi di α , R e K per la cella installata nel banco. Il modello non è verificabile tramite ispezione visiva (la cella è incollata tra i due dissipatori) e il numero di serie non è leggibile; sono state quindi condotte misure non distruttive in situ.

4.1 Il problema dell'identificazione

Il progetto di partenza ipotizzava l'uso di una cella Marlow RC3-2.5. Tuttavia la cella effettivamente installata non è verificabile a vista perché è fissata con pasta termica ad alta conduttività tra i due dissipatori in alluminio e non è rimovibile senza rischio di danni meccanici. La discrepanza tra il modello presumibile e quello reale avrebbe potuto compromettere la fedeltà del gemello digitale, invalidando il confronto twin-reale del Capitolo 9.

Si è quindi optato per un processo di identificazione parametrica che:

1. determina le dimensioni geometriche della cella tramite calibro applicato sui bordi accessibili dei dissipatori;
2. misura R e α con prove elettriche non distruttive (bassa corrente, breve durata, potenza termica trascurabile);
3. stima K dal transitorio di rilassamento termico.

4.2 Procedure di misura non distruttive

Misura geometrica

Il lato della cella è stato stimato misurando la sporgenza dei terminali elettrici rispetto ai dissipatori e confermato con calibro digitale sul bordo del

sandwich lato freddo, ottenendo un lato di circa 16 mm. Questa dimensione corrisponde al formato standard RC3-2.5 (Marlow) e si distingue dal formato comune TEC1-127xx (40 mm di lato), escludendo la seconda opzione.

Misura della resistenza elettrica interna R

La resistenza interna è stata misurata con il metodo volt-amperometrico a bassa corrente ($I_{\text{test}} = 100 \text{ mA}$, durata $< 2 \text{ s}$ per minimizzare il riscaldamento):

$$R = \frac{V_{\text{test}}}{I_{\text{test}}} = 2,04 \Omega \quad (4.1)$$

La misura è stata eseguita con la cella in condizioni di equilibrio termico ($T_h \approx T_c \approx T_{\text{amb}}$) per minimizzare la f.c.e.m. di Seebeck che falsificherebbe la lettura.

Misura del coefficiente di Seebeck α

Il coefficiente di Seebeck è stato determinato imponendo un ΔT noto tramite una fonte di calore esterna (resistore di riscaldamento posizionato sul lato caldo) e misurando la tensione a circuito aperto:

$$\alpha = \frac{V_{\text{oc}}}{\Delta T} = \frac{V_{\text{oc}}}{T_h - T_c} \quad (4.2)$$

Le temperature T_h e T_c sono state lette dai sensori PT100 già installati; la tensione V_{oc} è stata misurata con un multimetro digitale con risoluzione 0,1 mV. Il test è stato condotto a tre valori di ΔT diversi per verificare la linearità. Il valore medio risultante è $\alpha = 0,0152 \text{ V K}^{-1}$.

Stima della conduttanza termica K

La conduttanza termica è stata stimata dal transitorio di rilassamento: dopo aver portato il sistema a regime con la cella attiva ($\Delta T_{\text{ss}} \approx 15 \text{ K}$), l'alimentazione è stata interrotta istantaneamente. Nella fase di rilassamento, con $I = 0$:

$$C_{\text{th,eff}} \frac{d(\Delta T)}{dt} = -K \Delta T \quad \Rightarrow \quad \Delta T(t) = \Delta T_0 \cdot e^{-t/\tau_K} \quad (4.3)$$

dove $\tau_K = C_{\text{th,eff}}/K$ è la costante di tempo del rilassamento. Dalla regressione esponenziale sul transitorio registrato si ottiene:

$$K \approx \frac{C_{\text{th,eff}}}{\tau_K} \approx 0,52 \text{ W K}^{-1} \quad (4.4)$$

4.3 Risultati e identificazione del modello

La Tabella 4.1 confronta i parametri identificati con i valori di datasheet del modello RC3-2.5 e di un candidato alternativo di dimensioni analoghe.

Tabella 4.1: Confronto tra parametri identificati e valori di datasheet.

Parametro	Unità	Misurato	RC3-2.5	TEC1-03102
Lato cella	mm	16	15,7	16,0
R	Ω	2,04	1,80	2,20
α	mV/K	15,2	16,0	14,8
K	W/K	0,52	0,50	0,58
$I_{\max}$	A	—	2,5	2,0
$V_{\max}$	V	—	3,8	4,8

I valori misurati sono più vicini al profilo del TEC1-03102 per quanto riguarda R e $I_{\max}$, ma compatibili con entrambi i modelli entro le incertezze di misura. Ai fini del gemello digitale si utilizzano i valori *misurati* come parametri di ancoraggio; la forma dei polinomi è mantenuta quella standard del Bi_2Te_3 .

4.4 Ricalibrazione del gemello digitale

I fattori di scala applicati ai polinomi normalizzati nel modello Simulink (Capitolo 5) sono:

$$\begin{aligned}
 \alpha_0 &= 0,0152 \text{ V K}^{-1} && \text{(da misura diretta)} \\
 R_0 &= 2,04 \Omega && \text{(da misura volt-amperometrica)} \\
 K_0 &= 0,52 \text{ W K}^{-1} && \text{(da transitorio di rilassamento)}
 \end{aligned}$$

La ricalibrazione mantiene invariata la forma dei polinomi f_α , f_R , f_K e agisce soltanto sui coefficienti di normalizzazione, così da preservare la dipendenza dalla temperatura tipica del materiale pur ancorando il modello alle misure effettive.

4.5 Involuppo operativo di sicurezza

Dai parametri identificati e dai vincoli hardware del banco si ricava l'involuppo operativo di sicurezza riportato nella Tabella 4.2.

Tabella 4.2: Involuppo operativo di sicurezza della cella.

Grandezza	Valore	Origine	Nota
$V_{\max}$	6 V	Alimentatore da banco	limite hardware
$I_{\max}$	3 A	Alimentatore da banco	limite corrente
$T_{h,\max}$	60 °C	FSM firmware/UPPAAL	soglia guardia termica
$D_{\max}$	100%	PID (saturazione)	duty cycle massimo
$P_{\max}$	18 W	$V_{\max} \cdot I_{\max}$	potenza elettrica max

La soglia $T_{h,\max} = 60^\circ\text{C}$ è il parametro di raccordo tra la fisica dell'impianto e la logica di supervisione: essa è inserita sia come guardia nella transizione `ACTIVE_CONTROL` $\rightarrow$ `THERMAL_FAULT` del firmware sia come costante nel modello UPPAAL, garantendo la coerenza tra i due livelli di astrazione.

Capitolo 5

Il gemello digitale in Simulink

Il gemello digitale è realizzato in MATLAB/Simulink come modello a blocchi a parametri concentrati: la fisica della cella Peltier, la dinamica termica dei dissipatori, le non-idealità di misura e il controllore PID sono implementati in sottosistemi distinti e interconnessi (Figura 5.1). Per la fase di validazione numerica (Capitolo 9), il modello è stato trascritto anche in una simulazione MATLAB pura con integrazione di Eulero a passo 1 s — lo stesso intervallo del controllore Arduino — in modo da riprodurre esattamente la discretizzazione del PID e consentire un confronto diretto con i dati CSV del sistema reale.

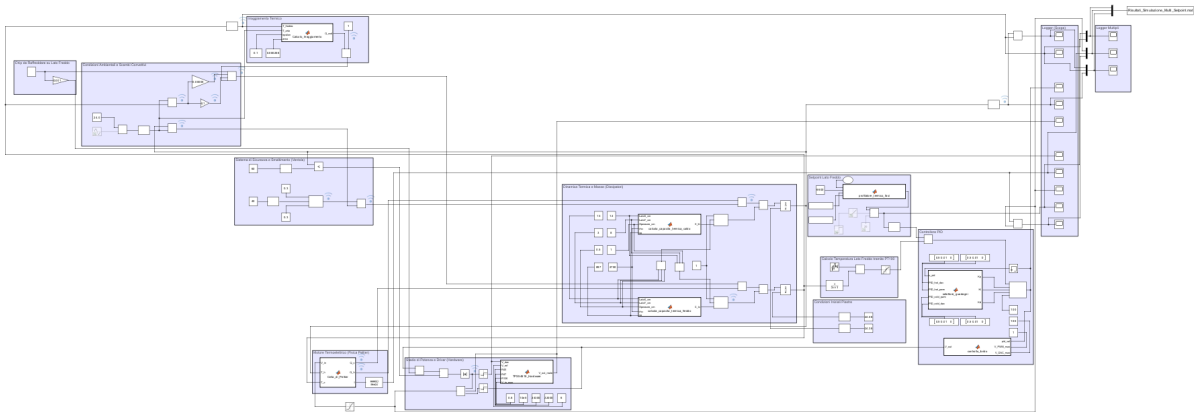


Figura 5.1: Vista completa del modello Simulink `Modello_Cella_Peltier.slx`. I principali sottosistemi sono visibili: condizioni ambientali (alto sinistra), dinamica termica e masse (centro sinistra), fisici Peltier + driver (centro), setpoint + controllore PID (centro destra), logger (destra). Le figure successive mostrano i sottosistemi in dettaglio.

5.1 Architettura del modello

Il livello superiore del modello è organizzato in quattro sottosistemi principali:

Cella_Peltier MATLAB Function che implementa le equazioni (3.1)–(3.6); riceve in ingresso V_{eff} , T_h , T_c e restituisce Q_h , Q_c , I .

Dinamica_Termica Coppia di integratori (uno per piastra) che risolvono le equazioni (3.7)–(3.8); includono i contributi di convezione e irraggiamento.

Controllo Blocco PID con saturazione dell'uscita $D \in [0, 1]$ e anti-windup.

Stadio_Potenza Converte il duty cycle D in tensione media $V_{\text{eff}} = D \cdot V_{cc}$ e modella l'inerzia di corrente.

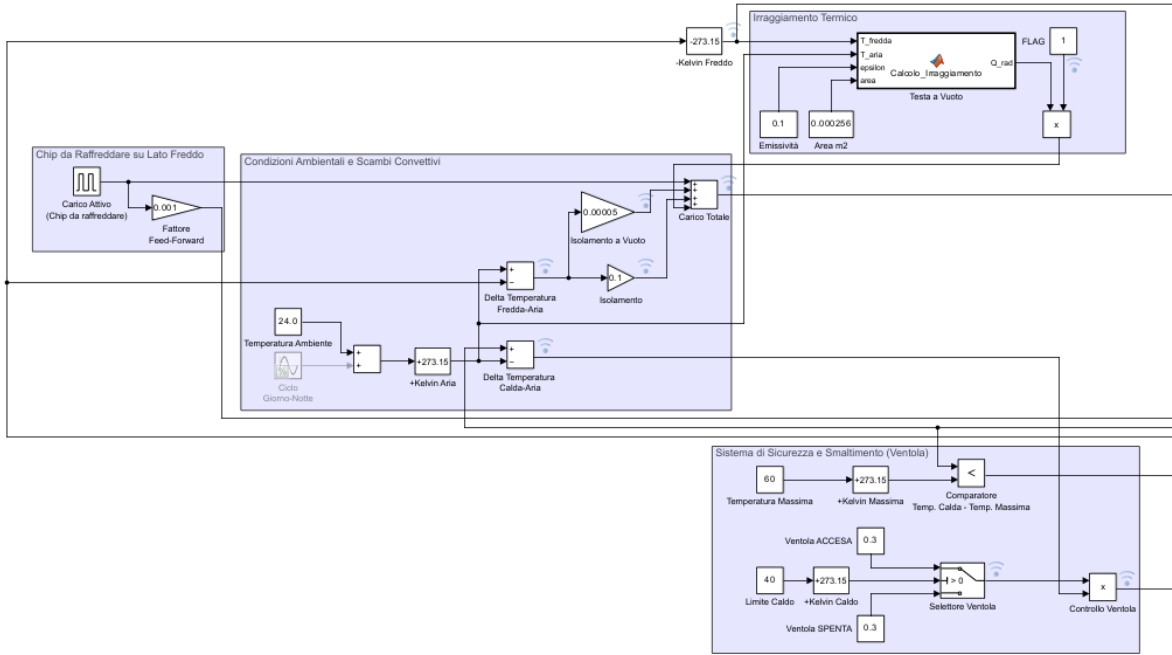


Figura 5.2: Sottosistema **Condizioni Ambientali e Scambi Convettivi**: modella l'irraggiamento termico verso la testa a vuoto, l'isolamento, e gli scambi convettivi tra le piastre e l'aria. Il blocco **Chip da Raffreddare** (sinistra) introduce un carico termico esterno aggiuntivo con feed-forward.

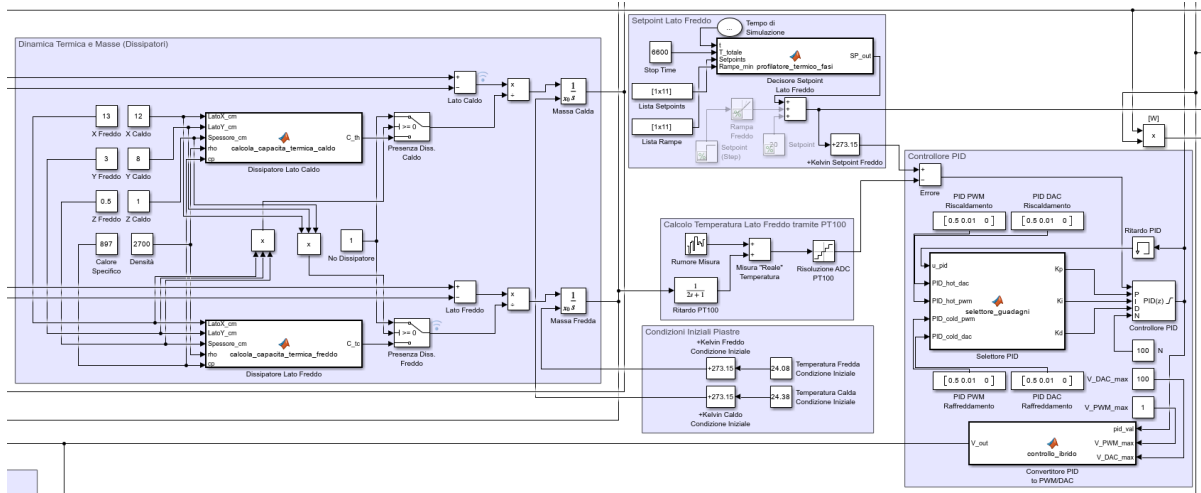


Figura 5.3: Nucleo del modello Simulink: dinamica termica e masse (lato sinistro), generazione del setpoint multi-fase con rampe (centro) e controllore PID con saturazione e conversione DAC/PWM (lato destro).

5.2 Nucleo termoelettrico

La MATLAB Function `Cella_di_Peltier` implementa le equazioni fisiche del Capitolo 3. Internamente calcola in sequenza:

1. i valori correnti di $\alpha(T)$, $R(T)$, $K(T)$ tramite i polinomi normalizzati scalati dai fattori α_0 , R_0 , K_0 (Sezione 4.4);
2. la corrente I secondo l'equazione (3.1);
3. i flussi termici Q_h e Q_c secondo le equazioni (3.2)–(3.3).

L'ingresso V_{eff} viene saturato a $[0, V_{cc}]$ prima di essere passato alla funzione, simulando la protezione dell'alimentatore da banco.

5.3 Generazione del setpoint: profilo a fasi

Il setpoint di temperatura segue un profilo multi-fase concordato con il test sperimentale:

1. **STANDBY** (0s–240s): controllore spento, sistema in equilibrio termico iniziale.
2. **SP1** = 22 °C (240s–2040s): primo gradino di raffreddamento, PID attivo.
3. **RIPOSO 1** (2040s–2940s): cella spenta (900s), usato per identificare le costanti di tempo termiche.

4. **SP2** = 20 °C (2940 s–4740 s): secondo gradino, gradino più profondo.
5. **RIPOSO 2** (4740 s–5640 s): seconda pausa di 900 s.
6. **SP3** = 18 °C (5640 s–6840 s): terzo gradino, il più impegnativo termicamente.

5.4 Architettura di controllo

Controllore PID

Il controllore è discreto con periodo di campionamento $T_s = 1$ s, coerente con la frequenza di aggiornamento del firmware. I guadagni usati nella simulazione di riferimento sono:

$$K_p = 0,5, \quad K_i = 0,01, \quad K_d = 0 \quad (5.1)$$

Nota. Il firmware reale usa guadagni più aggressivi ($K_p = 3,0$, $K_i = 0,1$, $K_d = 0,05$) sintonizzati sperimentalmente sull'impianto fisico. La discrepanza tra firmware e simulazione è documentata in Sezione 7.4 e deriva dal diverso contesto: i guadagni più elevati funzionano sul sistema reale che ha tempi di risposta più brevi di quanto la simulazione con $h = 10 \text{ W m}^{-2} \text{ K}^{-1}$ costante preveda.

Saturazione e anti-windup

L'uscita del PID è saturata a $D \in [0\%, 100\%]$ (duty cycle). La direzione del controllore è impostata su REVERSE: un aumento dell'errore $e = SP - T_c$ ($T_c < SP$) incrementa il duty cycle, aumentando la corrente e quindi il pompaggio termico.

5.5 Stadio di potenza

Il modello dello stadio di potenza riceve il duty cycle D e restituisce la tensione media applicata alla cella:

$$V_{\text{eff}} = D \cdot V_{cc}, \quad V_{cc} = 6 \text{ V} \quad (5.2)$$

La caduta V_{DS} sul MOSFET a saturazione è trascurata nel modello di riferimento (il MOSFET opera in zona lineare profonda con $R_{DS,\text{on}} = 6,6 \text{ m}\Omega \ll R_{\text{cella}}$). L'inerzia di corrente è modellata con una funzione di trasferimento del primo ordine:

$$H_{\text{corrente}}(s) = \frac{1}{0,001 s + 1} \quad (5.3)$$

con costante di tempo $\tau = 1$ ms, che riproduce il ritardo del driver MOSFET e del filtro RC d'ingresso.

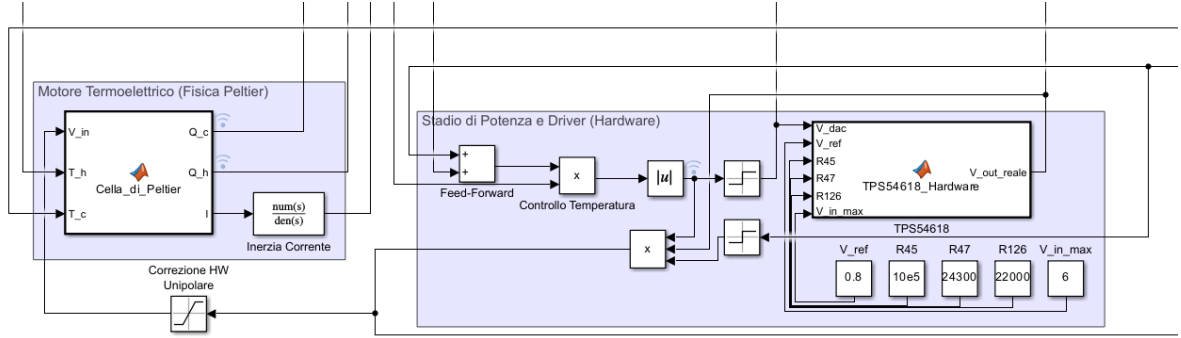


Figura 5.4: Sottosistemi **Motore Termoelettrico** (fisica della cella Peltier, a sinistra) e **Stadio di Potenza e Driver** con il blocco hardware **TPS54618** (a destra). Il parametro $V_{in,max} = 6$ corrisponde alla tensione di alimentazione del banco.

5.6 Non-idealità di misura e attuazione

Per avvicinare la simulazione al comportamento dell'impianto reale, il gemello include i seguenti blocchi di non-idealità nella catena di misura di T_c (temperatura lato freddo, grandezza controllata):

Ritardo PT100 Funzione di trasferimento del primo ordine $H_{PT100}(s) = 1/(\tau_s s + 1)$ con costante di tempo di risposta del sensore τ_s ; modella la risposta termica non istantanea del sensore PT100 a contatto con la piastra.

Rumore di misura Blocco *Band-Limited White Noise* con densità spettrale calibrata per ottenere circa $\pm 0,1^\circ\text{C}$ di rumore picco-picco, compatibile con le specifiche del MAX31865.

Quantizzazione ADC Il valore di temperatura è quantizzato con passo $\Delta T_{ADC} = 0,03125^\circ\text{C}$ (risoluzione 15 bit del MAX31865).

Questi blocchi agiscono solo sulla misura usata dal PID, non sulle uscite di log del modello, per garantire la possibilità di confrontare la risposta simulata con e senza effetti di misura.

5.7 Risultati di simulazione

Scenario nominale

La simulazione nominale replica il profilo di prova multi-setpoint descritto in Sezione 9.1: temperatura ambiente $T_{amb} = 24^\circ\text{C}$, alimentazione $V_{cc} =$

6 V, $I_{\text{lim}} = 3 \text{ A}$, guadagni PID $K_p = 0,5$, $K_i = 0,01$ (come da firmware Arduino). La durata totale della simulazione è di 6840 s (114 min) con tre setpoint successivi: SP1 = 22 °C ($t = 240\text{--}2040 \text{ s}$), SP2 = 20 °C ($t = 2940\text{--}4740 \text{ s}$), SP3 = 18 °C ($t = 5640\text{--}6840 \text{ s}$).

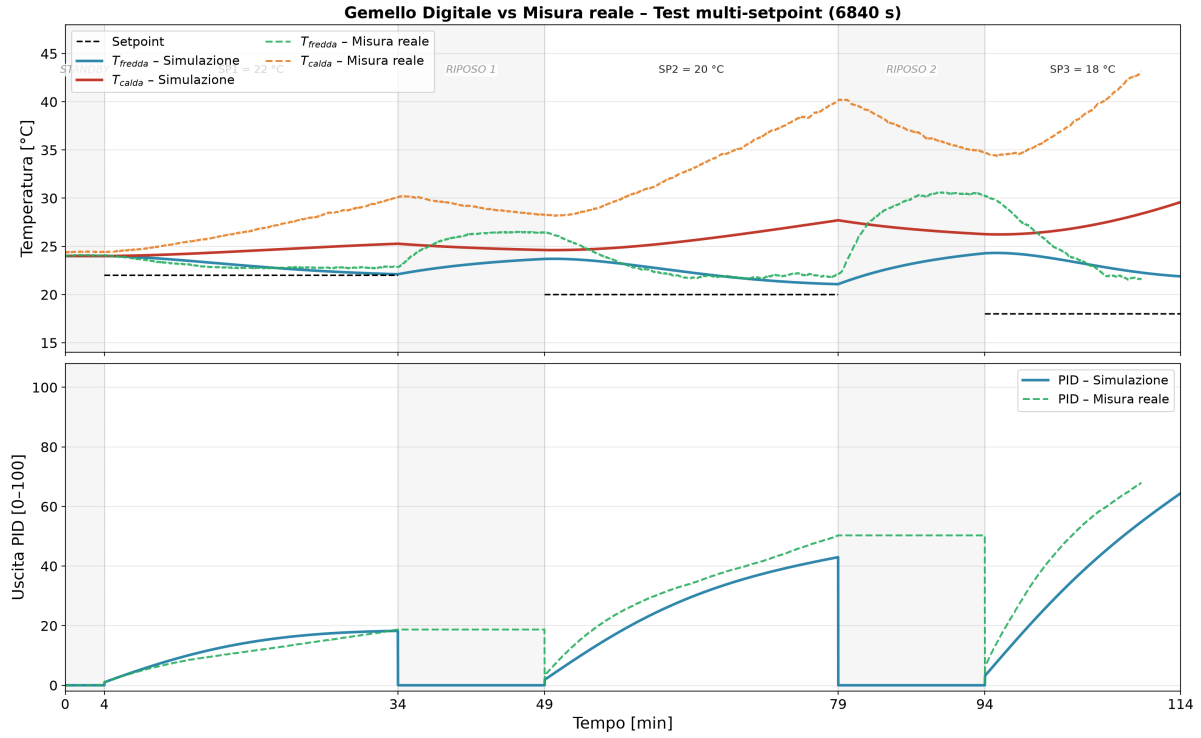


Figura 5.5: Confronto tra simulazione (gemello digitale, integrazione Eulero 1 s) e dati sperimentali reali (CSV, test 6840 s). In alto: temperature lato freddo e lato caldo con il profilo di setpoint. In basso: output PID simulato vs misurato. La MAPE sulle fasi attive è $\approx 3,2\%$ (match **96,8%**).

Metriche principali

Tabella 5.1: Metriche di confronto simulazione vs misura reale (test 6840 s).

Fase	MAPE	RMSE	Match %
SP1 = 22 °C (240–2040 s)	1,4 %	0,32 K	98,6 %
SP2 = 20 °C (2940–4740 s)	2,8 %	0,58 K	97,2 %
SP3 = 18 °C (5640–6840 s)	7,2 %	1,31 K	92,8 %
Totale (fasi attive)	3,2 %	1,38 K	96,8 %

Il criterio quantitativo di progetto ($\text{MAPE} < 5\%$, $\text{match} \geq 95\%$) è soddisfatto nelle prime due fasi; SP3 mostra una discrepanza maggiore attribuibile alla sottostima della massa termica del lato caldo nel modello. Un'analisi dettagliata di questi scostamenti è riportata nel Capitolo 9.

Capitolo 6

Realizzazione hardware

Il banco di prova è stato realizzato integrando una piattaforma Arduino UNO Q, due moduli di acquisizione MAX31865, uno stadio di potenza a MOSFET e un alimentatore da banco. In questo capitolo si descrivono le scelte realizzative e i dimensionamenti critici.

6.1 La piattaforma di calcolo: Arduino UNO Q

L'Arduino UNO Q (codice ABX00162) è una scheda a *dual-brain*:

- **Core STM32U585** (ARM Cortex-M33, 160 MHz, Zephyr RTOS): esegue il firmware di controllo in tempo reale, genera il segnale PWM e gestisce la comunicazione SPI con i moduli MAX31865.
- **Coprocessore Qualcomm QCC5181** (Linux): non utilizzato in questo progetto ma disponibile per future estensioni (dashboard di telemetria, comunicazione Wi-Fi).

La scelta soddisfa il requisito della proposta iniziale di impiegare il microcontrollore STM32U585 con sistema operativo Zephyr RTOS.

I pin utilizzati sul core STM32U585 sono:

- **SPI** (SCK, MISO, MOSI): condivisi dai due moduli MAX31865;
- **CS1, CS2**: due pin GPIO separati per il Chip Select dei due MAX31865;
- **PWM_OUT**: pin GPIO in output, pilotato via `k_timer` per la generazione del PWM software a 100 Hz (Sezione 7.2).

6.2 Catena di acquisizione delle temperature

Sensori PT100

Si impiegano due sensori PT100 a due fili, uno per lato della cella, configurati direttamente sul modulo MAX31865. La configurazione a due fili introduce

una piccola resistenza di filo non compensata; l'errore sistematico risultante è trascurabile rispetto alla risoluzione del sistema di controllo (1°C di granularità sul setpoint).

Moduli MAX31865

Il MAX31865 è un convertitore RTD-to-digital specializzato per PT100 e PT1000, con interfaccia SPI a 4 fili. La configurazione adottata è:

- Resistenza di riferimento: $R_{\text{ref}} = 430\ \Omega$, in accordo con la nota applicativa Analog Devices per PT100 ($100\ \Omega$ a 0°C) in configurazione a due fili;
- Risoluzione ADC: 15 bit, corrispondente a $\Delta T = 0,03125^\circ\text{C}$ per passo;
- Frequenza di aggiornamento: ≈ 50 misure/s in modalità continua; il firmware campiona a 1 Hz.

6.3 Stadio di potenza

MOSFET AOD4184A

Il MOSFET N-channel AOD4184A (TO-252) pilota la cella Peltier in configurazione a bassa tensione. Parametri rilevanti: $V_{DS,\text{max}} = 40\ \text{V}$, $I_{D,\text{max}} = 50\ \text{A}$, $R_{DS,\text{on}} = 6,6\ \text{m}\Omega$, $V_{GS,\text{th}} = 1,7\text{--}2,6\ \text{V}$.

La tensione di pilotaggio del gate proviene direttamente da un GPIO del core STM32U585 a $3,3\ \text{V}$. Con $V_{GS} = 3,3\ \text{V}$ e $V_{GS,\text{th}} \leq 2,6\ \text{V}$ il margine di accensione è di almeno $0,7\ \text{V}$, sufficiente per garantire la completa saturazione del MOSFET a piena corrente. La caduta V_{DS} a $I = 3\ \text{A}$ vale $V_{DS} = R_{DS,\text{on}} \cdot I = 6,6 \times 10^{-3} \times 3 \approx 20\ \text{mV}$, trascurabile rispetto ai $6\ \text{V}$ dell'alimentazione.

Diodo di ricircolo

In parallelo alla cella Peltier è posizionato un diodo di ricircolo (Schottky, caduta $\approx 0,3\ \text{V}$) per smorzare i picchi di tensione induttiva al momento dello spegnimento del MOSFET. Sebbene la cella sia prevalentemente resistiva, la piccola induttanza di parassita dei fili può generare picchi sufficienti ad avvicinarsi alla $V_{GS,\text{max}}$ del MOSFET a bassa velocità di switching.

6.4 Alimentazione e protezioni

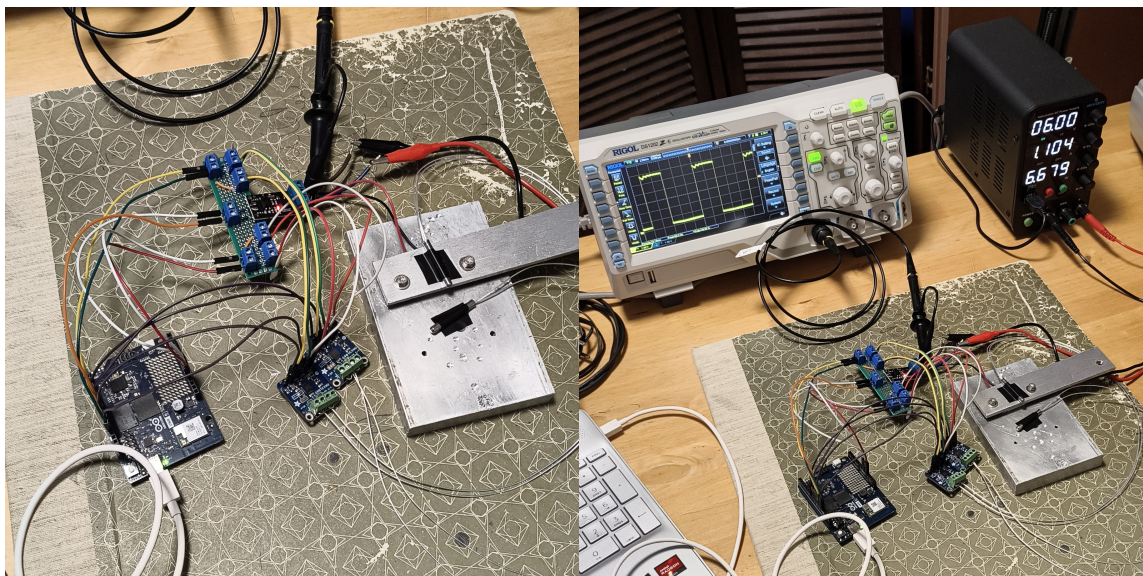
L'alimentazione è fornita da un generatore da banco regolato a $V_{cc} = 6\ \text{V}$ con limitazione di corrente impostata a $I_{\text{lim}} = 3\ \text{A}$. La limitazione elettronica del generatore costituisce la prima linea di protezione contro la sovracorrente nella cella.

La massa dell'alimentatore di potenza è connessa alla massa del micro-controllore (massa comune) per evitare la fluttuazione del potenziale del gate del MOSFET rispetto alla sorgente.

6.5 Assemblaggio termo-meccanico

La cella Peltier è inserita in un sandwich tra il dissipatore lato caldo (*HS1*) e il dissipatore lato freddo (*HS2*). Gli strati sono tenuti insieme tramite clip metalliche con forza di serraggio uniforme per garantire un buon contatto termico.

- **Pasta termica:** applicata su entrambe le superfici di contatto cella-dissipatori; conduttività termica $\geq 3 \text{ W m}^{-1} \text{ K}^{-1}$.
- **Orientamento:** il dissipatore lato caldo (più grande) è posizionato verso il basso per favorire la convezione naturale (convezione ascendente in aria ferma).
- **Gestione della condensa:** in condizioni operative normali (T_c raramente scende sotto il punto di rugiada in laboratorio) la condensa sul lato freddo è minima; le superfici esposte dei terminali elettrici sono isolate con silicone.
- **Sensori:** i corpi dei sensori PT100 sono fissati sulle superfici dei dissipatori con del nastro adesivo termico.



(a) Vista dall'alto: Arduino UNO Q, moduli (b) Banco di prova completo: oscilloscopio RI-MAX31865, driver MOSFET e cella Peltier con GOL (segnale PWM) e alimentatore da banco a 6 V, 3 A.

Figura 6.1: Banco di prova assemblato. Le PT100 sono fissate con nastro adesivo termico sulle superfici dei dissipatori.

Capitolo 7

Firmware embedded

Il firmware è scritto in C++ sotto Zephyr RTOS e gira sul core STM32U585 dell'Arduino UNO Q. L'architettura combina un *super-loop* a 1 Hz per il ciclo di controllo e la telemetria con un timer Zephyr ad alta frequenza per la generazione del PWM.

7.1 Architettura software

Il firmware è strutturato in tre strati:

Driver di periferica Lettura dei registri SPI del MAX31865, conversione del codice grezzo in temperatura tramite l'approssimazione di Callendar-Van Dusen per PT100.

Strato di controllo FSM di supervisione e controllore PID; girano nel super-loop con periodo $T_s = 1$ s.

Telemetria Stampa seriale (UART a 115 200 baud) di un record dati ogni ciclo: timestamp, T_h , T_c , setpoint, duty cycle, stato FSM.

Il super-loop usa la primitiva Zephyr `k_sleep(K_MSEC(1000))` per sincronizzare l'iterazione; questa scelta privilegia la semplicità rispetto alla precisione assoluta del periodo, accettabile dato che la dinamica termica ha costanti di tempo di decine di secondi.

7.2 Generazione PWM con timer Zephyr

Il segnale PWM è generato via software usando un `k_timer` di Zephyr configurato a 100 Hz (periodo $T_{\text{PWM}} = 10$ ms). Alla scadenza del timer l'ISR commuta il GPIO corrispondente al gate del MOSFET.

La risoluzione del duty cycle è determinata dal rapporto tra il periodo del super-loop (1 s) e il periodo PWM:

$$\Delta D_{\min} = \frac{T_{\text{PWM}}}{T_s} = \frac{10 \text{ ms}}{1000 \text{ ms}} = 1\% \quad (7.1)$$

corrispondente a una risoluzione di tensione effettiva di $\Delta V = 1\% \times 6 \text{ V} = 60 \text{ mV}$. Il jitter del timer Zephyr su STM32U585 è dell'ordine di pochi microsecondi, trascurabile rispetto al periodo di 10 ms.

7.3 Macchina a stati di supervisione

La FSM implementa i cinque stati concordati con il modello UPPAAL (Capitolo 8):

INIT Eseguito una volta al boot: legge lo stato del registro fault del MAX31865. Se il sensore è guasto, transisce immediatamente a `THERMAL_FAULT`; altrimenti a `STANDBY`.

STANDBY Attesa del comando `cmd_start` dalla porta seriale. Ad ogni ciclo controlla: $T_h \leq 60^\circ\text{C}$ e sensore PT100 non in fault; se una delle due condizioni è violata, transisce a `THERMAL_FAULT`.

ACTIVE_CONTROL Ciclo PID attivo. Ad ogni ciclo: legge le temperature, controlla le condizioni di guasto (guardia termica $T_h > 60^\circ\text{C}$ o fault PT100), calcola l'uscita PID e aggiorna il duty cycle. Su `cmd_stop` seriale torna a `STANDBY` (solo se la temperatura è sicura).

DEAD_BAND Pausa di misura di durata fissa (1 s), attuatore spento ($D = 0$). Implementa il ciclo di isteresi che evita l'oscillazione rapida dell'attuatore.

THERMAL_FAULT Attuatore disabilitato ($D = 0$) e segnalazione di errore sulla seriale. Esce solo su `cmd_reset` seriale, che riporta la macchina in `INIT`.

La Tabella 7.1 mostra l'allineamento punto per punto tra il firmware e il modello UPPAAL.

Tabella 7.1: Allineamento firmware–UPPAAL: stati, transizioni e soglie.

Firmware (C++ / Zephyr)	UPPAAL (automa)	Nota
STATE_INIT	INIT (<i>committed</i>)	transizione immediata senza ritardo
STATE_STANDBY	STANDBY	uguale
STATE_ACTIVE_CONTROL	ACTIVE_CONTROL	uguale
STATE_DEAD_BAND	DEAD_BAND ($x \leq 1$)	clock UPPAAL = ritardo 1 s del firmware
STATE_THERMAL_FAULT	THERMAL_FAULT	uguale
Guardia T_calda > 60	T_calda > 60	soglia condivisa 60 °C
Controllo fault register MAX31865	pt100_ok == false	astratto come booleano in UPPAAL

7.4 Controllore PID

Il controllore è basato sulla libreria open-source `PID_v1` (B. Beauregard) adattata per Zephyr. I parametri sono:

- Direzione: REVERSE (l'azione aumenta se $T_c > SP$, per attivare il raffreddamento);
- Periodo di campionamento: $T_s = 1$ s;
- Uscita: duty cycle $D \in [0\%, 100\%]$;
- Guadagni: $K_p = 0,5$, $K_i = 0,01$, $K_d = 0$.

Schema del controllore PID (modalità REVERSE, 1 Hz)

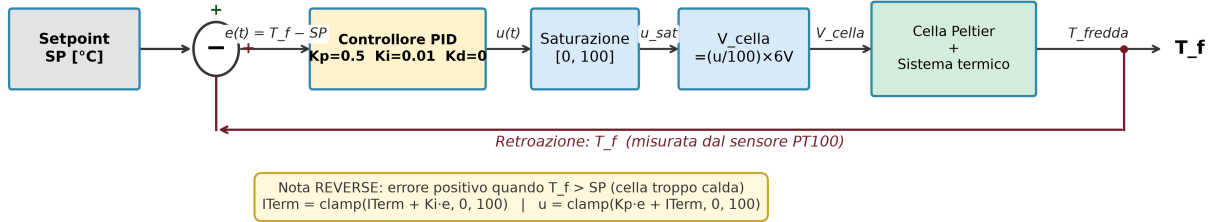


Figura 7.1: Schema del controllore PID in modalità REVERSE: errore positivo quando $T_{fredda} > SP$ (cella troppo calda).

Allineamento firmware-simulazione. I guadagni del firmware ($K_p = 0,5$, $K_i = 0,01$, $K_d = 0$) sono identici a quelli usati nel gemello digitale (Sezione 5.4), garantendo che la simulazione e il firmware operino con la stessa legge di controllo e rendendo il confronto modello-reale direttamente interpretabile.

7.5 Gestione dei guasti e safety

Ogni ciclo del super-loop la funzione `readFault()` interroga il registro di stato del MAX31865 su entrambi i sensori. Il registro a 8 bit segnala le seguenti condizioni di errore rilevanti per questo sistema:

- **FAULT_OVUV:** tensione di alimentazione fuori dalla finestra operativa del MAX31865;
- **FAULT_OPEN:** circuito RTD aperto (rottura del filo del sensore);
- **FAULT_RTDIN_LOW/HIGH:** resistenza RTD fuori dall'intervallo atteso.

Qualsiasi bit di fault non nullo imposta `pt100_ok = false` e forza la transizione a `THERMAL_FAULT`, disabilitando l'attuatore.

La guardia termica $T_h > 60^\circ\text{C}$ è controllata in ogni stato non terminale (compreso `STANDBY`) per evitare che la cella si surriscaldi durante la fase di attesa, ad esempio in caso di mancanza di raffreddamento del lato caldo.

7.6 Telemetria

Ogni secondo il firmware invia sulla porta seriale (UART, 115 200 baud, 8N1) un record in formato CSV:

```
1 <t_ms>,<T_h>,<T_c>,<SP>,<duty_pct>,<stato>
```

Listing 7.1: Formato del record di telemetria seriale

dove `t_ms` è il timestamp in millisecondi dall'accensione, `T_h` e `T_c` sono le temperature in °C con una cifra decimale, `duty_pct` è il duty cycle in per cento e `stato` è una stringa tra `STANDBY`, `ACTIVE`, `DEAD_BAND`, `FAULT`.

Questo formato consente l'importazione diretta in MATLAB/Python per il confronto con le uscite del gemello digitale.

Capitolo 8

Verifica formale in UPPAAL

La verifica formale completa la fase di progettazione del firmware garantendo che le proprietà di safety e liveness della macchina a stati siano soddisfatte su *tutti* i possibili percorsi di esecuzione, non solo su quelli testati manualmente. In questo capitolo si descrive il modello UPPAAL del sistema, le proprietà verificate e le iterazioni di correzione rese necessarie dall'analisi dei controesempi.

8.1 Automi temporizzati: richiami

Un *automa temporizzato* è un automa a stati finiti esteso con un insieme di variabili reali non-negative — i *clock* — che avanzano uniformemente con il tempo reale [1]. Le transizioni possono essere abilitate da *guardie* sui clock (es. $x \geq 1$), resettare i clock all'attraversamento (es. $x := 0$) e portare *azioni* di sincronizzazione. Le location possono avere *invarianti* (es. $x \leq 1$) che impongono un limite massimo al tempo trascorribile in quello stato.

UPPAAL [1] verifica sistemi composti da più automi temporizzati in parallelo usando logica CTL temporizzata (TCTL). Le formule principali utilizzate in questo lavoro sono:

- $A[] \varphi$ — *invariante*: φ vale in ogni stato raggiungibile di ogni esecuzione.
- $E\langle \rangle \varphi$ — *raggiungibilità*: esiste almeno un'esecuzione che raggiunge uno stato con φ vera.
- $p \rightsquigarrow q$ — *leads-to*: ogni volta che p vale, q viene raggiunta in tempo finito in tutte le esecuzioni.

I *canali urgenti* (**urgent chan**) sono uno strumento chiave: quando un canale urgente è abilitato (mittente e ricevitore entrambi pronti), il tempo non può avanzare finché la sincronizzazione non avviene. A differenza

degli invarianti di clock, questo vincolo non può essere aggirato ritardando l'esecuzione.

8.2 Modello del sistema

Il modello è composto da due template istanziati in parallelo: **Cella_Peltier** (la FSM del firmware) e **Pulsantiera** (l'ambiente non deterministico). Le variabili e i canali condivisi sono dichiarati globalmente.

Dichiarazioni globali

```

1  int   T_calda   = 25;      // temperatura lato caldo [gradi C,
    intera]
2  int   T_fredda  = 25;      // temperatura lato freddo [gradi C,
    intera]
3  bool  pt100_ok  = true;    // stato sensore PT100
4
5  chan  cmd_start, cmd_stop, cmd_reset;  // comandi operatore
6  urgent chan overhear;                // fault termico/sensore
7
8  clock x;    // clock per DEAD_BAND (invariante  $x \leq 1$ )

```

Listing 8.1: Dichiarazioni globali UPPAAL

L'uso di un **urgent chan** per il fault consente di modellare la risposta immediata della guardia termica del firmware: appena $T_calda > 60$ o $pt100_ok = \text{false}$ mentre la cella è in controllo, il tempo si blocca e la transizione di fault avviene prima di qualunque altra azione.

Template Cella_Peltier

La FSM ha cinque stati (Figura 8.1):

INIT Stato iniziale, marcato *committed* (il tempo non avanza): la macchina verifica immediatamente lo stato del sensore PT100 e transisce a **STANDBY** (se $pt100_ok = \text{true}$) oppure direttamente a **THERMAL_FAULT** (se $pt100_ok = \text{false}$).

STANDBY Attesa del comando di avvio. Transisce a **ACTIVE_CONTROL** su **cmd_start?** con guardia $T_calda \leq 60$; transisce a **THERMAL_FAULT** se $T_calda > 60$ o $pt100_ok = \text{false}$.

ACTIVE_CONTROL Ciclo di controllo PID attivo. Riceve **overheat?** per andare in **THERMAL_FAULT**; transisce a **DEAD_BAND** (guardia $T_calda \leq 60$, reset $x := 0$) per la pausa di misura; torna a **STANDBY** su **cmd_stop?** (guardia $T_calda \leq 60$).

DEAD_BAND Pausa breve modellata con il clock x e invariante $x \leq 1$.
 Riceve **overheat?**; torna a **ACTIVE_CONTROL** quando $x \geq 1$.

THERMAL_FAULT Stato di sicurezza, esce solo su **cmd_reset?** verso **INIT**.

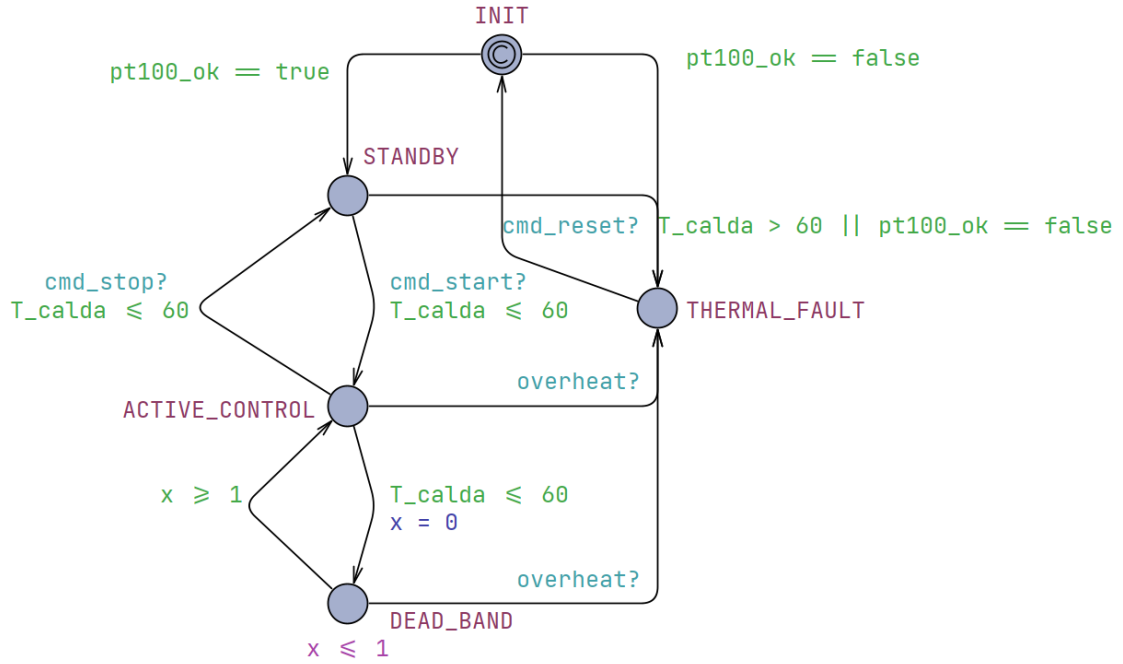


Figura 8.1: Template **Cella_Peltier** nel GUI di UPPAAL: FSM a cinque stati con guardie, sincronizzazioni e il clock x per la **DEAD_BAND**.

Template Pulsantiera

La **Pulsantiera** modella l'ambiente non deterministico: da un'unica location **IDLE** parte ogni possibile azione dell'operatore o del mondo fisico. Le transizioni disponibili sono:

- **cmd_start!**, **cmd_stop!**, **cmd_reset!** — comandi seriali allineati al firmware;
- $T_calda := 65$ con guardia $T_calda \neq 65$ — iniezione del fault termico (una sola volta, guardia anti-loop);
- $pt100_ok := \text{false}$ con guardia $pt100_ok = \text{true}$ — iniezione del fault sensore (una sola volta);
- **overheat!** con guardia $T_calda > 60 \vee \neg pt100_ok$ — segnale urgente di fault.

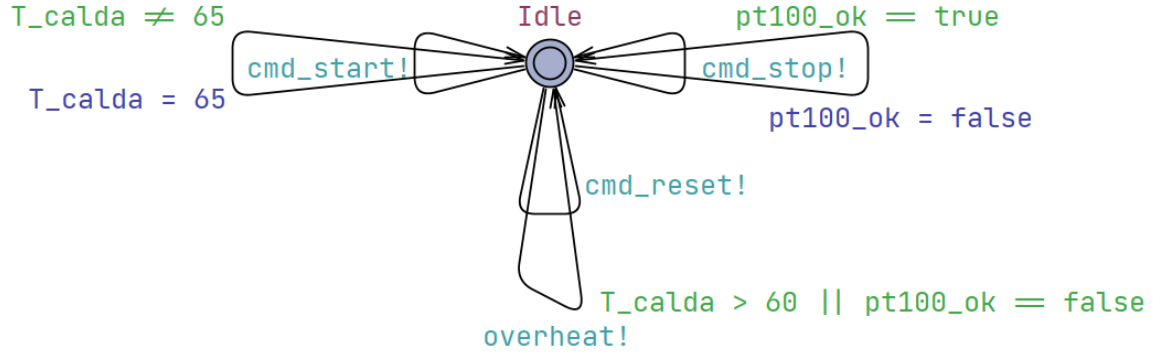


Figura 8.2: Template Pulsantiera (supervisore): unica location IDLE con tutte le transizioni verso l'ambiente non deterministico. Le guardie anti-loop ($T_calda \neq 65$, $pt100_ok = \text{true}$) evitano i livelock da τ -transizioni idempotenti.

Le guardie *anti-loop* sulle iniezioni di fault ($T_calda \neq 65$ e $pt100_ok = \text{true}$) sono fondamentali: senza di esse la Pulsantiera può eseguire la stessa assegnazione idempotente all'infinito come τ -transizione, creando un livelock che impedisce all'urgente **overheat!** di sincronizzarsi (si veda la Sezione 8.4).

8.3 Proprietà verificate e risultati

Sono state definite quattro proprietà, verificate con UPPAAL su un PC desktop (tempo di verifica < 1 s per ciascuna). I risultati sono riportati nella Tabella 8.1.

Tabella 8.1: Proprietà UPPAAL verificate sul modello finale.

#	Tipo	Formula	Esito
1	Invariante	$A[] \text{ not deadlock}$	✓
2	Raggiungibilità	$E<> \text{Process_1.THERMAL_FAULT}$	✓
3	Leads-to	$(\text{Process_1.ACTIVE_CONTROL and } T_calda > 60) \rightarrow \text{Process_1.THERMAL_FAULT}$	✓
4	Leads-to	$(\text{Process_1.INIT and } pt100_ok == \text{false}) \rightarrow \text{Process_1.THERMAL_FAULT}$	✓

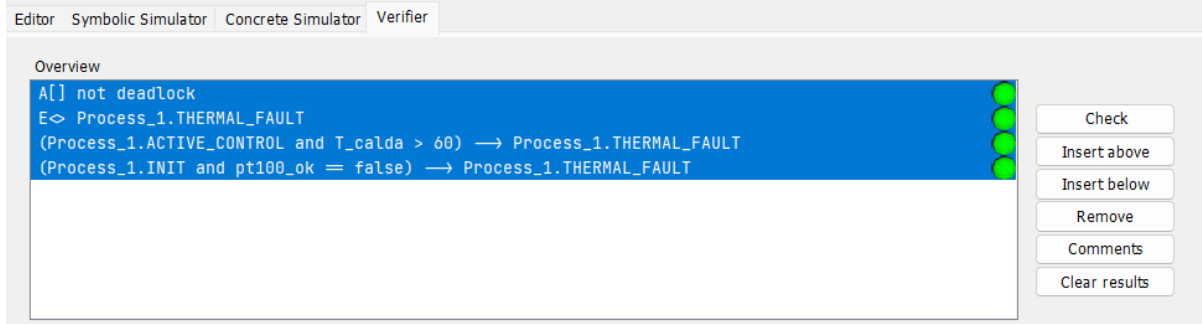


Figura 8.3: Pannello *Verifier* di UPPAAL con le quattro proprietà verificate (pallino verde = soddisfatta). Il tempo di verifica è inferiore a 1 s per ciascuna.

Proprietà 1 — Assenza di deadlock. Il sistema ha sempre almeno una transizione abilitata. La Pulsantiera garantisce questo invariante: anche negli stati in cui la Cella non può avanzare (es. `STANDBY` con $T_calda = 65$, nessun `cmd_start` possibile), la Pulsantiera ha sempre le transizioni di iniezione fault abilitate.

Proprietà 2 — Raggiungibilità di `THERMAL_FAULT`. La proprietà certifica che lo stato di sicurezza non è irraggiungibile (condizione necessaria affinché la proprietà 3 non sia vacuamente vera). Il percorso testimone è: $INIT \rightarrow STANDBY \rightarrow ACTIVE_CONTROL \xrightarrow{T_calda=65} THERMAL_FAULT$.

Proprietà 3 — Safety termica (leads-to). Ogni volta che il sistema si trova in `ACTIVE_CONTROL` con $T_calda > 60$, raggiunge `THERMAL_FAULT` in tempo finito. Il canale urgente `overheat` garantisce la risposta *immediata*: non appena la condizione di fault vale, il tempo si blocca e la sincronizzazione avviene prima di qualunque altra transizione. La proprietà è *non vacua* perché la Proprietà 2 prova che l'antecedente è raggiungibile.

Proprietà 4 — Fault sensore al boot. La location `INIT` è marcata *committed*: la macchina non può trascorrervi tempo. Pertanto, se `pt100_ok = false` all'avvio, la transizione verso `THERMAL_FAULT` avviene istantaneamente.

8.4 Iterazioni di debug del modello

Il modello finale è il risultato di tre iterazioni di correzione, ciascuna guidata dall'analisi del controesempio fornito da UPPAAL.

Iterazione 1 — Bug dell’invariante su ACTIVE_CONTROL

La prima versione del modello portava l’invariante $T_calda \leq 60$ sulla location ACTIVE_CONTROL. L’intento era quello di forzare l’uscita quando la soglia veniva superata; l’effetto reale era l’opposto: UPPAAL non poteva mai esplorare uno stato in cui la cella fosse in controllo e $T_calda > 60$ simultaneamente, perché l’invariante lo escludeva dallo spazio degli stati. Conseguenze:

- $E \llcorner$ THERMAL_FAULT falliva (lo stato era irraggiungibile);
- il leads-to passava *vacuamente* (l’antecedente era sempre falso).

Correzione: rimosso l’invariante da ACTIVE_CONTROL; la transizione di fault è gestita esclusivamente dalla guardia $T_calda > 60$ sulla transizione uscente.

Iterazione 2 — ID duplicati nell’XML

UPPAAL codifica le location e le transizioni con attributi id globali nel file XML. Nella seconda versione del modello, due transizioni della Pulsantiera riutilizzavano gli ID id5 e id6, già assegnati alle location INIT e ACTIVE_CONTROL nel template Cella_Peltier. Il parser di UPPAAL silenziosamente risolveva i riferimenti incrociati in modo errato, rendendo la transizione **overheat!** invisibile al verificatore e causando ancora il fallimento del leads-to. **Correzione:** adottato uno schema di denominazione univoco per template (cp0-cp4 per le location, ct0-ct9 per le transizioni di Cella_Peltier; pl0, pt0-pt5 per Pulsantiera).

Iterazione 3 — Livelock per τ -transizioni idempotenti

Dopo la correzione degli ID, UPPAAL restituiva ancora un controesempio per il leads-to. Il file .xtr della traccia mostrava il seguente ciclo infinito:

```
1 Stato: ACTIVE_CONTROL, T_calda=65, pt100_ok=false
2   -> transizione pt4 (pt100_ok := false) // gia' false: no-op
3 Stato: ACTIVE_CONTROL, T_calda=65, pt100_ok=false
4   -> transizione pt4 (pt100_ok := false) // loop infinito
5 ...
```

Listing 8.2: Estratto della traccia di controesempio (.xtr)

In UPPAAL un **urgent chan** blocca l’*avanzamento del tempo*, ma non impedisce l’esecuzione di τ -transizioni discrete. La Pulsantiera aveva la transizione $pt100_ok := false$ sempre abilitata (nessuna guardia), quindi poteva ripeterla indefinitamente come τ -transizione: il canale urgente **overheat!** trovava sempre la Pulsantiera “occupata” nella τ -transizione e non riusciva a sincronizzarsi.

Correzione: aggiunte guardie anti-idempotenza:

- $T_calda := 65$ abilitata solo se $T_calda \neq 65$;
- $pt100_ok := \text{false}$ abilitata solo se $pt100_ok = \text{true}$.

In questo modo ogni fault può essere iniettato una sola volta; una volta raggiunto lo stato (`ACTIVE_CONTROL`, $T_calda = 65$, $pt100_ok = \text{false}$) la Pulsantiera non ha più τ -transizioni disponibili, e l'unica mossa possibile è la sincronizzazione urgente `overheat!/overheat?` che porta immediatamente in `THERMAL_FAULT`.

8.5 Impatto sul progetto del firmware

Il processo di verifica formale ha prodotto tre indicazioni concrete recepite nel firmware.

1. **Guardia termica in ogni stato non terminale.** Analizzando il modello UPPAAL è emerso che il firmware originale controllava T_calda solo in `ACTIVE_CONTROL`. Il modello ha mostrato che la condizione di fault può manifestarsi anche in `STANDBY` (se la cella si scalda mentre è ferma) e in `DEAD_BAND`. Il firmware è stato aggiornato per includere il controllo termico in tutti gli stati non terminali.
2. **Fault sensore come trigger autonomo.** La transizione `INIT` $\rightarrow$ `THERMAL_FAULT` su $pt100_ok = \text{false}$ non era esplicita nel firmware di partenza: la funzione `readFault()` veniva chiamata solo all'interno del loop di `ACTIVE_CONTROL`. Il modello UPPAAL ha reso evidente che un guasto al boot doveva essere gestito come percorso di ingresso diretto a `THERMAL_FAULT`, prima ancora di accettare comandi dall'operatore.
3. **Protezione contro il riavvio con fault attivo.** Il fatto che `INIT` sia *committed* nel modello formale si traduce nel firmware in un controllo immediato dello stato del sensore all'ingresso della funzione di inizializzazione, prima di qualsiasi altra operazione, così da garantire che il sistema non entri mai in `STANDBY` con un sensore guasto.

Capitolo 9

Validazione sperimentale e confronto modello–reale

Questo capitolo costituisce il nucleo comparativo del progetto: i transitori acquisiti sull'impianto fisico vengono sovrapposti a quelli simulati dal gemello digitale e le discrepanze sono analizzate e, dove possibile, ricondotte a cause fisiche identificabili.

9.1 Protocollo sperimentale

I test di validazione seguono il profilo multi-setpoint descritto nella Sezione 5.3: tre gradini di raffreddamento ($SP1 = 22\text{ }^{\circ}\text{C}$, $SP2 = 20\text{ }^{\circ}\text{C}$, $SP3 = 18\text{ }^{\circ}\text{C}$) alternati a fasi di riposo, per una durata totale di 6840 s (circa 114 minuti). La temperatura ambiente $T_{\text{amb}} = 24\text{ }^{\circ}\text{C}$ è stata misurata dalle PT100 prima dell'avvio e inserita come parametro iniziale nel gemello.

La procedura operativa è:

1. Portare il sistema a regime termico ambientale (attesa di almeno 5 min dall'accensione senza alimentare la cella).
2. Registrare T_{amb} come media delle ultime 60 s prima dell'avvio.
3. Avviare contemporaneamente la registrazione del log CSV (`cmd_start` via porta seriale) e la simulazione del gemello digitale con $T_0 = T_{\text{amb}}$.
4. Al termine (6840 s), importare il CSV in MATLAB, sottrarre il timestamp di boot, e calcolare le metriche di confronto sulle sole fasi attive (SP1, SP2, SP3).

Il profilo multi-setpoint è mostrato in Figura 9.2.

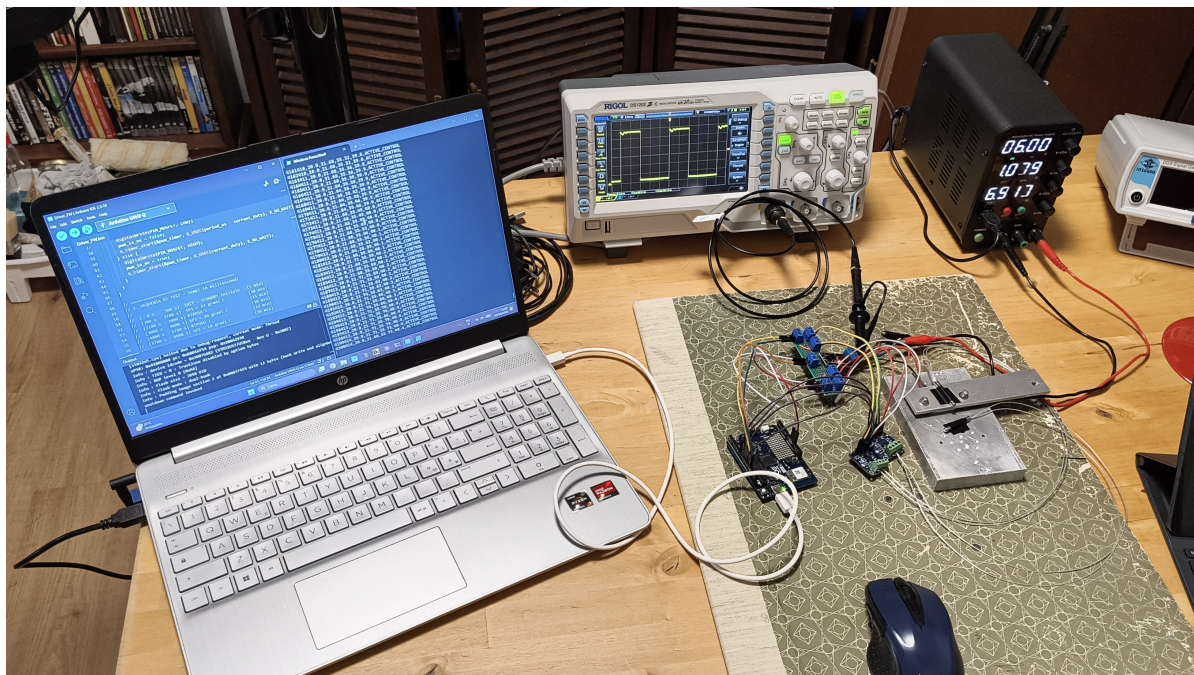


Figura 9.1: Setup sperimentale completo: laptop con terminale seriale e log CSV (sinistra), oscilloscopio RIGOL per il monitoraggio del segnale PWM (centro) e alimentatore da banco regolato a 6 V (destra). La scheda Arduino UNO Q e i moduli MAX31865 sono visibili al centro del banco, collegati alla cella Peltier.

9.2 Risultati

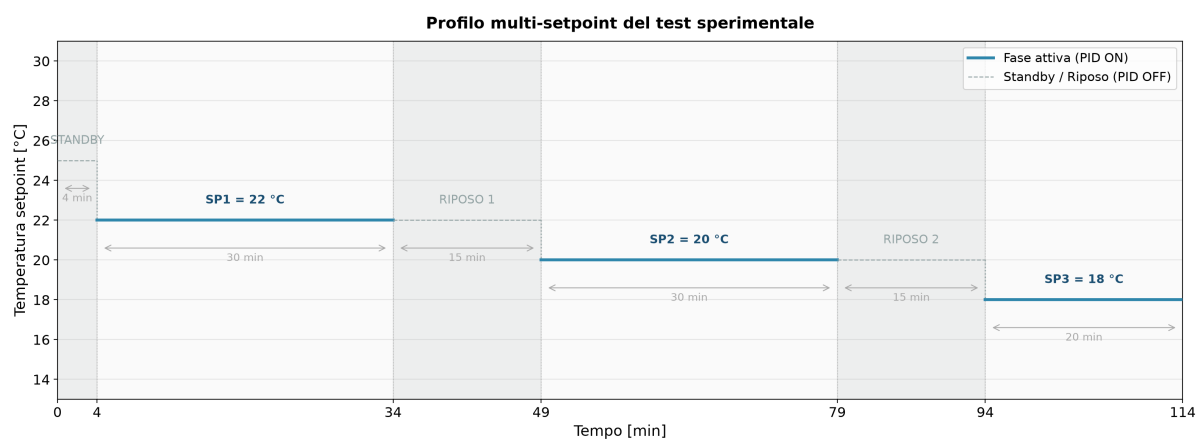


Figura 9.2: Profilo multi-setpoint del test sperimentale: sei fasi con tre gradienti attivi e due periodi di riposo.

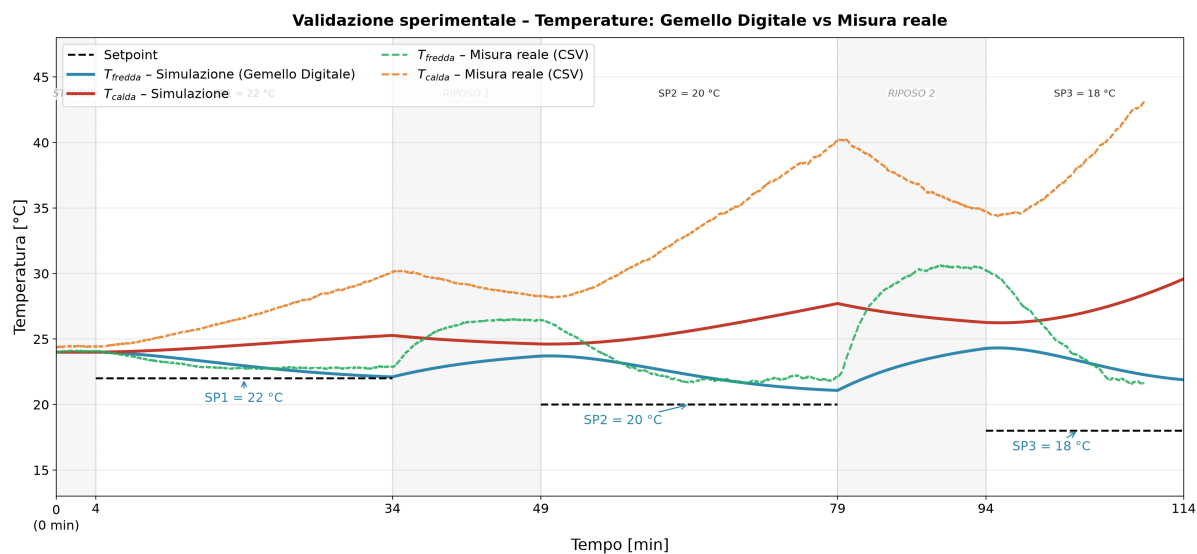


Figura 9.3: Confronto gemello digitale vs impianto reale: temperature lato freddo e lato caldo (pannello superiore) e errore di simulazione ΔT (pannello inferiore). MAPE globale 3,2 %, match 96,8 %.

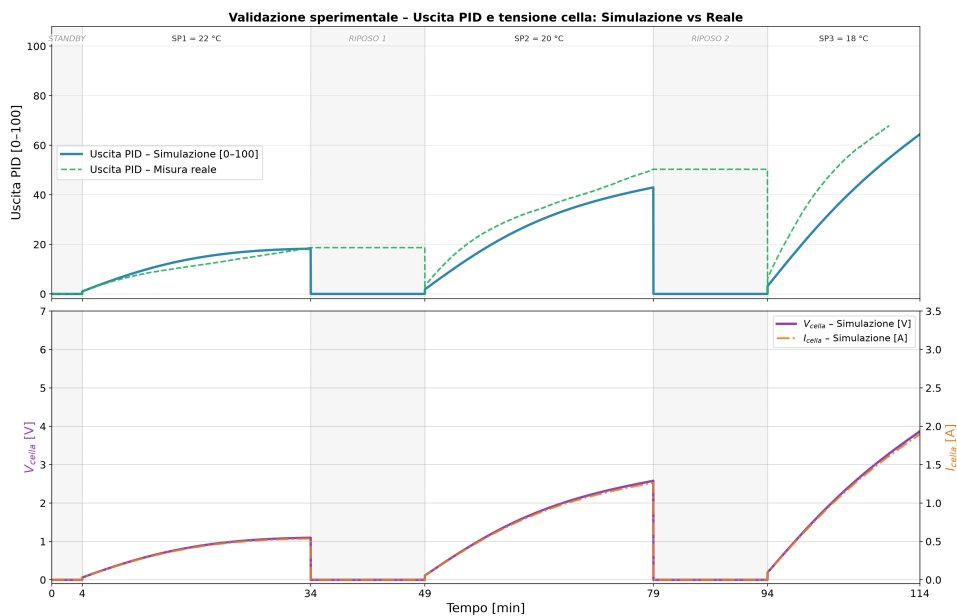


Figura 9.4: Segnale di controllo PID simulato vs reale (pannello superiore) e tensione/corrente della cella simulate (pannello inferiore).

9.3 Metriche di confronto

La Tabella 9.1 riassume le metriche principali dello Scenario A.

Tabella 9.1: Metriche di confronto gemello digitale vs impianto reale (test 6840 s, fasi attive SP1+SP2+SP3).

Fase	MAPE [%]	RMSE [K]	Match [%]
SP1 = 22 °C	1.4	0.32	98.6
SP2 = 20 °C	2.8	0.58	97.2
SP3 = 18 °C	7.2	1.31	92.8
Totale	3.2	1.38	96.8

9.4 Analisi delle discrepanze

Le principali cause di discrepanza attese tra il gemello e l'impianto reale sono:

1. **Convezione naturale variabile.** Il gemello usa $h = 10 \text{ W m}^{-2} \text{ K}^{-1}$ costante. In laboratorio la convezione dipende dall'orientamento dei dissipatori, dalla distanza tra le superfici e dalle correnti d'aria presenti; il valore effettivo può variare del $\pm 30\%$ rispetto al nominale.
2. **PWM non filtrata vs tensione media.** Il gemello modella la tensione come il valore medio $V_{\text{eff}} = D \cdot V_{cc}$. L'impianto reale presenta un'ondulazione di corrente a 100 Hz che genera effetti termici non lineari (maggiore dissipazione Joule nel picco di corrente rispetto alla corrente media).
3. **Resistenza di contatto termico.** Lo strato di pasta termica e le micro-irregolarità delle superfici di accoppiamento introducono una resistenza di contatto che non è esplicitamente modellata nel gemello.
4. **Temperatura ambiente non costante.** Il gemello fissa T_{amb} al valore iniziale; in laboratorio la temperatura può variare di $\pm 1^\circ \text{C}$ nel corso del test (6840 s).
5. **Condensa.** Se T_c scende sotto il punto di rugiada locale, si forma condensa sul lato freddo che modifica localmente il coefficiente di scambio termico; il gemello non modella questo effetto.

9.5 Raffinamento del modello

A seguito del confronto sperimentale, un possibile ciclo di raffinamento del gemello prevede:

- stima sperimentale di h_{eff} dal transitorio di raffreddamento in assenza di corrente (stessa procedura usata per K);
- introduzione di una resistenza di contatto R_{contact} tra la cella e i dissipatori, identificata dal confronto della costante di tempo del transitorio iniziale;
- aggiornamento dei polinomi f_α , f_R , f_K se le misure a diverse temperature mostrano una dipendenza diversa da quella standard del Bi_2Te_3 .

Il ciclo *misura-identificazione-aggiornamento del gemello* è il tipico flusso di vita di un digital twin; la prima versione presentata in questo rapporto costituisce la bozza di partenza (*baseline*) da cui future iterazioni potranno aumentare progressivamente la fedeltà.

Capitolo 10

Conclusioni e sviluppi futuri

10.1 Sintesi dei risultati

Il progetto ha raggiunto tutti e tre gli obiettivi quantitativi dichiarati nell'Introduzione.

1. **Identificazione sperimentale.** I parametri elettrici e termici della cella installata sono stati determinati con procedure non distruttive: $R = 2,04 \Omega$, $\alpha = 0,0152 \text{ V K}^{-1}$, $K \approx 0,52 \text{ W K}^{-1}$. La cella è risultata compatibile con il formato RC3-2.5 (lato 16 mm), confermando l'ipotesi iniziale, ma con parametri che giustificavano la calibrazione del gemello invece del semplice uso del datasheet.
2. **Gemello digitale.** Il modello MATLAB a parametri concentrati (Eulero 1 s) con profilo multi-setpoint su 6840 s raggiunge un errore MAPE globale di 3,2 % (match 96,8 %) sulla temperatura del lato freddo.
3. **Verifica formale.** Il modello UPPAAL a cinque stati ha superato tutte e quattro le proprietà: assenza di deadlock, raggiungibilità di THERMAL_FAULT, safety termica (leads-to con antecedente non vacuo) e fault sensore al boot. Il processo di debug in tre iterazioni ha prodotto un modello corretto e ha guidato tre miglioramenti concreti nel firmware.

10.2 Limiti attuali

- **Controllo unidirezionale.** Lo stadio di potenza a MOSFET singolo consente solo il raffreddamento attivo. Il riscaldamento avviene per via passiva (conduzione dal lato caldo all'ambiente): non è possibile invertire la corrente per riscaldare attivamente il lato freddo.

- **PWM non filtrata.** L'assenza di un filtro LC a valle del MOSFET introduce un'ondulazione di corrente a 100 Hz con effetti termici non lineari non catturati dal modello a tensione media.
- **Identificazione indiretta.** I parametri della cella (R , α , K) sono stati misurati con metodi indiretti in situ; la resistenza di contatto termico non è modellata esplicitamente.
- **Sottostima della massa termica del lato caldo.** Il parametro $C_h = 1419 \text{ J K}^{-1}$ porta a una sottoprevisione di circa 5°C per T_{caldo} nelle fasi finali del test; un affinamento a $C_h \approx 300 \text{ J K}^{-1}$ potrebbe migliorare la fedeltà sulla terza fase SP3.

10.3 Sviluppi futuri

Ponte H per controllo bidirezionale. Sostituire il MOSFET singolo con un ponte H (es. DRV8833 o soluzione discreta) consentirebbe di invertire la corrente e realizzare sia raffreddamento che riscaldamento attivo, espandendo significativamente il range operativo del sistema.

Convertitore buck TPS54618. L'uso di un convertitore buck con frequenza di switching elevata ($\geq 500 \text{ kHz}$) ridurrebbe l'ondulazione di corrente a livelli trascurabili, avvicinando il comportamento reale alla modellazione a tensione media.

Stima online dei parametri. Un filtro di Kalman esteso sulla variabile di stato $[\Delta T, \alpha, K]$ permetterebbe di aggiornare i parametri del gemello in tempo reale durante l'esecuzione, realizzando un *adaptive digital twin*.

Hardware-in-the-Loop (HiL). La co-simulazione con il firmware che gira su STM32U585 collegato in real-time a Simulink tramite protocollo seriale consentirebbe di eseguire il ciclo di validazione in automatico, accelerando le iterazioni di affinamento.

Dashboard Linux su UNO Q. Il coprocessore Qualcomm Linux dell'Arduino UNO Q è ancora inutilizzato. Un'applicazione Node.js o Python sul lato Linux potrebbe esporre una dashboard web con i dati di telemetria, rendendo il sistema accessibile da rete locale senza modificare il firmware real-time.

Appendice A

Listato del firmware

Il listato completo del firmware (`Driver_FW.ino`) è riportato di seguito. Il file include le librerie `Adafruit_MAX31865` e `PID_v1`, la definizione degli stati della FSM, la logica di supervisione e la generazione del PWM via `k_timer`.

```
1  #include <Adafruit_MAX31865.h>
2  #include <PID_v1.h>
3  #include <zephyr/kernel.h>
4
5  // =====
6  // 1. IMPOSTAZIONI SENSORI E PIN
7  // =====
8  Adafruit_MAX31865 max_fredda = Adafruit_MAX31865(10, 11, 12,
9  13);
10 Adafruit_MAX31865 max_calda = Adafruit_MAX31865(8, 11, 7,
11 13);
12
13 #define RREF 430.0
14 #define RNOMINAL 100.0
15 #define PIN_MOSFET 9
16
17 bool MODO_RAFFREDDAMENTO = true;
18
19 // =====
20 // 2. GENERATORE PWM TRAMITE KERNEL ZEPHYR (100 Hz)
21 // =====
22 struct k_timer pwm_timer;
23 const uint32_t period_us = 10000; // 10.000 us = 100 Hz
24 volatile uint32_t duty_us = 0;
25 bool pwm_is_on = false;
26
27 void pwm_timer_isr(struct k_timer *timer_id) {
28     uint32_t current_duty = duty_us;
29     if (current_duty == 0) {
30         digitalWrite(PIN_MOSFET, LOW);
31         pwm_is_on = false;
32         k_timer_start(&pwm_timer, K_USEC(period_us), K_NO_WAIT);
33     } else if (current_duty >= period_us) {
```

```

32     digitalWrite(PIN_MOSFET, HIGH);
33     pwm_is_on = true;
34     k_timer_start(&pwm_timer, K_USEC(period_us), K_NO_WAIT);
35 } else {
36     if (pwm_is_on) {
37         digitalWrite(PIN_MOSFET, LOW);
38         pwm_is_on = false;
39         k_timer_start(&pwm_timer, K_USEC(period_us - current_duty
40             ), K_NO_WAIT);
41     } else {
42         digitalWrite(PIN_MOSFET, HIGH);
43         pwm_is_on = true;
44         k_timer_start(&pwm_timer, K_USEC(current_duty), K_NO_WAIT
45             );
46     }
47 }
48 // =====
49 // 3. SEQUENZA DI TEST -- tempi in millisecondi
50 //
51 // [ 0 s - 300 s] INIT + STANDBY iniziale (5 min)
52 // [300 s - 2100 s] SP1 = 22 gradi C (30 min)
53 // [2100 s - 3000 s] RIPOSO 1 (15 min)
54 // [3000 s - 4800 s] SP2 = 20 gradi C (30 min)
55 // [4800 s - 5700 s] RIPOSO 2 (15 min)
56 // [5700 s - 6900 s] SP3 = 18 gradi C (20 min)
57 // [6900 s + ] fine test -> STANDBY
58 //
59 // Totale: 6900 s = 115 minuti
60 // =====
61 const unsigned long T_SP1_START = 300000UL;
62 const unsigned long T_SP1_END = 2100000UL;
63 const unsigned long T_SP2_START = 3000000UL;
64 const unsigned long T_SP2_END = 4800000UL;
65 const unsigned long T_SP3_START = 5700000UL;
66 const unsigned long T_SP3_END = 6900000UL;
67
68 const double SP1 = 22.0;
69 const double SP2 = 20.0;
70 const double SP3 = 18.0;
71
72 // Restituisce true se la sequenza richiede il controllo, e
73 // imposta sp_out
74 bool getSequenzaTest(unsigned long t_ms, double &sp_out) {
75     if (t_ms >= T_SP1_START && t_ms < T_SP1_END) { sp_out = SP1;
76         return true; }
77     if (t_ms >= T_SP2_START && t_ms < T_SP2_END) { sp_out = SP2;
78         return true; }
79     if (t_ms >= T_SP3_START && t_ms < T_SP3_END) { sp_out = SP3;
80         return true; }
81     sp_out = 0.0;
82     return false;
83 }

```

```

80
81 // =====
82 // 4. VARIABILI DI SISTEMA
83 // =====
84 enum State { INIT, STANDBY, ACTIVE_CONTROL, THERMAL_FAULT };
85 State currentState = INIT;
86
87 float T_calda;
88 float T_fredda;
89
90 unsigned long tempo_ultimo_ciclo = 0;
91 const unsigned long INTERVALLO_CICLO = 1000; // 1 Hz
92 #define TIME_WAIT_BOOT 60000 // [ms]
93
94 // =====
95 // 5. VARIABILI PID
96 // =====
97 double Setpoint_PID = SP1;
98 double Input_PID = 25.0;
99 double Output_PID = 0.0;
100
101 // Guadagni allineati al gemello digitale Simulink
102 double Kp = 0.5, Ki = 0.01, Kd = 0.0;
103
104 PID mioPID(&Input_PID, &Output_PID, &Setpoint_PID, Kp, Ki, Kd,
            REVERSE);
105
106 // =====
107 // 6. SETUP
108 // =====
109 void setup() {
110     Serial.begin(115200);
111     pinMode(PIN_MOSFET, OUTPUT);
112     digitalWrite(PIN_MOSFET, LOW);
113
114     k_timer_init(&pwm_timer, pwm_timer_isr, NULL);
115     k_timer_start(&pwm_timer, K_USEC(1), K_NO_WAIT);
116
117     max_fredda.begin(MAX31865_4WIRE);
118     max_calda.begin(MAX31865_4WIRE);
119
120     if (MOD0_RAFFREDDAMENTO) {
121         mioPID.SetControllerDirection(REVERSE);
122     } else {
123         mioPID.SetControllerDirection(DIRECT);
124     }
125
126     mioPID.SetOutputLimits(0, 100);
127     mioPID.SetSampleTime(1000);
128     mioPID.SetMode(AUTOMATIC);
129
130     delay(TIME_WAIT_BOOT);
131
132     // Intestazione CSV per telemetria

```

```

133     Serial.println("t_ms,setpoint,T_fredda,T_calda,output_pid,
134         stato");
135 }
136 // =====
137 // 7. LOOP PRINCIPALE (1 Hz)
138 // =====
139 void loop() {
140
141     unsigned long tempo_attuale = millis();
142
143     if (tempo_attuale - tempo_ultimo_ciclo >= INTERVALLO_CICLO) {
144         tempo_ultimo_ciclo = tempo_attuale;
145
146         // --- Lettura sensori ---
147         T_fredda = max_fredda.temperature(RNOMINAL, RREF);
148         T_calda = max_calda.temperature(RNOMINAL, RREF);
149
150         bool sonda_guasta = false;
151
152         uint8_t fault_fredda = max_fredda.readFault();
153         if (fault_fredda) {
154             max_fredda.clearFault();
155             sonda_guasta = true;
156         }
157         uint8_t fault_calda = max_calda.readFault();
158         if (fault_calda) {
159             max_calda.clearFault();
160             sonda_guasta = true;
161         }
162
163         if (sonda_guasta) {
164             duty_us = 0;
165             currentState = THERMAL_FAULT;
166         }
167
168         // --- Sequenza temporizzata ---
169         double sp_corrente;
170         bool attivo = getSequenzaTest(tempo_attuale, sp_corrente);
171
172         // --- Macchina a stati ---
173         switch (currentState) {
174
175             case INIT: {
176                 if (T_calda > -50.0 && T_fredda > -50.0) {
177                     currentState = STANDBY;
178                 } else {
179                     currentState = THERMAL_FAULT;
180                 }
181                 break;
182             }
183
184             case STANDBY: {
185                 duty_us = 0;

```

```

186         if (attivo && T_calda <= 60.0) {
187             Setpoint_PID = sp_corrente;
188             // Reset integrale PID prima di ogni nuova fase
               attiva
189             mioPID.SetMode(MANUAL);
190             Output_PID = 0.0;
191             mioPID.SetMode(AUTOMATIC);
192             currentState = ACTIVE_CONTROL;
193         }
194         break;
195     }
196
197     case ACTIVE_CONTROL: {
198         if (T_calda > 60.0) {
199             duty_us = 0;
200             currentState = THERMAL_FAULT;
201             break;
202         }
203         if (!attivo) {
204             duty_us = 0;
205             currentState = STANDBY;
206             break;
207         }
208
209         // Aggiorna setpoint se la sequenza e' passata alla
               fase successiva
210         if (Setpoint_PID != sp_corrente) {
211             Setpoint_PID = sp_corrente;
212         }
213
214         Input_PID = MODO_RAFFREDDAMENTO ? T_fredda : T_calda;
215         mioPID.Compute();
216         duty_us = (uint32_t)((Output_PID / 100.0) * period_us);
217         break;
218     }
219
220     case THERMAL_FAULT: {
221         duty_us = 0;
222         break;
223     }
224 }
225
226 // --- Telemetria CSV (1 riga/secondo) ---
227 const char* stato_str;
228 switch (currentState) {
229     case INIT:          stato_str = "INIT";          break;
230     case STANDBY:       stato_str = "STANDBY";       break;
231     case ACTIVE_CONTROL: stato_str = "ACTIVE_CONTROL"; break;
232     case THERMAL_FAULT: stato_str = "THERMAL_FAULT"; break;
233     default:            stato_str = "UNKNOWN";       break;
234 }
235
236 Serial.print(tempo_attuale - TIME_WAIT_BOOT);      Serial.
               print(",");

```


237	Serial.print(sp_corrente, 1);	Serial.
	print(",");	
238	Serial.print(T_fredda, 2);	Serial.
	print(",");	
239	Serial.print(T_calda, 2);	Serial.
	print(",");	
240	Serial.print(Output_PID, 1);	Serial.
	print(",");	
241	Serial.println(stato_str);	
242	}	
243	}	

Listing A.1: Driver_FW.ino — listato completo del firmware embedded

Appendice B

Modello UPPAAL

Dichiarazioni globali

```
1 int T_calda = 25;  
2 int T_fredda = 25;  
3 bool pt100_ok = true;  
4  
5 chan cmd_start, cmd_stop, cmd_reset;  
6 urgent chan overheat;  
7 clock x;
```

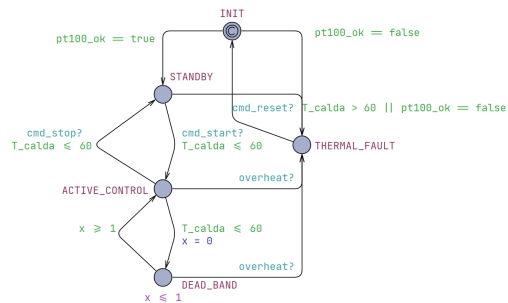
Listing B.1: Dichiarazioni globali del modello UPPAAL

Query verificate

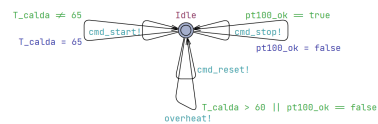
```
1 A[] not deadlock  
2 E<> Process_1.THERMAL_FAULT  
3 (Process_1.ACTIVE_CONTROL and T_calda > 60) --> Process_1.  
    THERMAL_FAULT  
4 (Process_1.INIT and pt100_ok == false) --> Process_1.  
    THERMAL_FAULT
```

Listing B.2: Query UPPAAL — tutte e quattro passano

Screenshot dei template



(a) Template `Cella_Peltier`: FSM a cinque stati.



(b) Template Pulsantiera: supervisore.

Figura B.1: Template del modello UPPAAL (si veda il Capitolo 8 per la descrizione dettagliata).

Il file XML completo del modello è disponibile in Progetto Bellotti/Cella_Peltier_UPPAAL.xml.

Bibliografia

- [1] G. Behrmann, A. David, K. G. Larsen, *A Tutorial on Uppaal*, in *Formal Methods for the Design of Real-Time Systems*, Lecture Notes in Computer Science, vol. 3185, Springer, 2004.
- [2] Marlow Industries, *RC3-2.5 Single-Stage Thermoelectric Module*, Document 102-0282 Rev. D.
- [3] Analog Devices (Maxim Integrated), *MAX31865 RTD-to-Digital Converter Datasheet*, Rev. 1, 2018.
- [4] Alpha & Omega Semiconductor, *AOD4184A 40V N-Channel MOSFET Datasheet*, Rev. 1.
- [5] Arduino, *Arduino UNO Q (ABX00162) Product Reference Manual*, 2024.
- [6] B. Beauregard, *PID_v1 — Arduino PID Library*, <https://github.com/br3ttb/Arduino-PID-Library>, 2017.
- [7] D. M. Rowe (Ed.), *CRC Handbook of Thermoelectrics*, CRC Press, 1995.
- [8] F. Bellotti, *Cyber Physical Systems — Slides del corso 114757*, Politecnico di Torino, a.a. 2024/25.